```
cloudra/tools/core/xtb_frames.py:160    # ---------------------------------------------------------------------------
cloudra/tools/core/xtb_frames.py:161
cloudra/tools/core/xtb_frames.py:162    _EIGVAL_LINE_RE = re.compile(r"^\s*eigval\s*:\s*(.*)$")
cloudra/tools/core/xtb_frames.py:163    # 虚频的另一种写法：数值带 i 后缀（如 "68.66i"），语义等同负值
cloudra/tools/core/xtb_frames.py:164    _EIGVAL_TOKEN_RE = re.compile(rf"^{_FLOAT}(i?)$", re.IGNORECASE)
cloudra/tools/core/xtb_frames.py:165
cloudra/tools/core/xtb_frames.py:166
cloudra/tools/core/xtb_frames.py:167    def _eigval_tokens(line_body: str) -> list[float]:
cloudra/tools/core/xtb_frames.py:168        values: list[float] = []
cloudra/tools/core/xtb_frames.py:169        for tok in line_body.split():
cloudra/tools/core/xtb_frames.py:170            m = _EIGVAL_TOKEN_RE.match(tok)
cloudra/tools/core/xtb_frames.py:171            if m is None:
cloudra/tools/core/xtb_frames.py:172                continue
cloudra/tools/core/xtb_frames.py:173            value = float(m.group(1).replace("D", "E").replace("d", "e"))
cloudra/tools/core/xtb_frames.py:174            values.append(-abs(value) if m.group(2) else value)
cloudra/tools/core/xtb_frames.py:175        return values
cloudra/tools/core/xtb_frames.py:176
cloudra/tools/core/xtb_frames.py:177
cloudra/tools/core/xtb_frames.py:178    def vibfreqs_from_text(text: str) -> list[float]:
cloudra/tools/core/xtb_frames.py:179        """解析最后一个 projected vibrational frequencies 块的频率（cm⁻¹）。
cloudra/tools/core/xtb_frames.py:180
cloudra/tools/core/xtb_frames.py:181        负值=虚频（i 后缀写法同样解析为负值）；保留打头的平动/转动零频模式
cloudra/tools/core/xtb_frames.py:182        （与 orca_frames 同款语义），调用方按符号拆分实频/虚频即可。无频率块
cloudra/tools/core/xtb_frames.py:183        时返回空列表——调用方须保持 None 语义，不得当成零虚频。
cloudra/tools/core/xtb_frames.py:184        """
cloudra/tools/core/xtb_frames.py:185        idx = text.rfind("projected vibrational frequencies")
cloudra/tools/core/xtb_frames.py:186        if idx < 0:
cloudra/tools/core/xtb_frames.py:187            return []
cloudra/tools/core/xtb_frames.py:188        freqs: list[float] = []
cloudra/tools/core/xtb_frames.py:189        for line in text[idx:].splitlines()[1:]:
cloudra/tools/core/xtb_frames.py:190            m = _EIGVAL_LINE_RE.match(line)
cloudra/tools/core/xtb_frames.py:191            if m is None:
cloudra/tools/core/xtb_frames.py:192                if freqs:
cloudra/tools/core/xtb_frames.py:193                    break  # eigval 行连续成块，块外即停
cloudra/tools/core/xtb_frames.py:194                continue
cloudra/tools/core/xtb_frames.py:195            freqs.extend(_eigval_tokens(m.group(1)))
cloudra/tools/core/xtb_frames.py:196        return freqs
cloudra/tools/core/xtb_frames.py:197
cloudra/tools/core/xtb_frames.py:198
cloudra/tools/core/xtb_frames.py:199    # ---------------------------------------------------------------------------
cloudra/tools/core/xtb_frames.py:200    # 任务类型识别
cloudra/tools/core/xtb_frames.py:201    # ---------------------------------------------------------------------------
cloudra/tools/core/xtb_frames.py:202
cloudra/tools/core/xtb_frames.py:203    # opt 周期分隔行 "............................. CYCLE    8 ..........."
cloudra/tools/core/xtb_frames.py:204    _OPT_CYCLE_RE = re.compile(r"^\.{3,}\s*CYCLE\s+\d+\s*\.{3,}\s*$", re.M)
cloudra/tools/core/xtb_frames.py:205
cloudra/tools/core/xtb_frames.py:206
cloudra/tools/core/xtb_frames.py:207    def detect_job_types_from_text(text: str) -> set[str]:
cloudra/tools/core/xtb_frames.py:208        """识别 xtb 日志的任务类型，返回 {"opt","freq","sp"} 的子集。
cloudra/tools/core/xtb_frames.py:209
cloudra/tools/core/xtb_frames.py:210        xtb stdout 不回显 CLI 标志，只能从输出节判定：CYCLE 迭代行/收敛标记
cloudra/tools/core/xtb_frames.py:211        → opt；projected vibrational frequencies 块 → freq；非优化即 sp。
```

```
        TS 搜索无独立输出节，判不出（xtb 的 saddle 优化走外部约束文件）。
        """
        jobs: set[str] = set()
        if (
            _OPT_CONVERGED_MARKER in text
            or _OPT_NOT_CONVERGED_MARKER in text
            or _OPT_CYCLE_RE.search(text)
        ):
            jobs.add("opt")
        if "projected vibrational frequencies" in text:
            jobs.add("freq")
        if "opt" not in jobs:
            jobs.add("sp")
        return jobs
"""xTB MTD 路径搜索运行器：TS 初猜的便宜端（SOP `guess` 阶段）。

**事实核对（本机 xtb 6.7.1 实测，不是推断）**

- xTB **没有 NEB**。它有 ``--path [FILE]``：元动力学（MTD）路径搜索，从输入几何走到
  给定的产物几何。所以本模块叫 path 不叫 neb。
- 调用形式：``xtb reactant.xyz --path product.xyz``。**两个几何必需**，元素序列必须一致。
- 产出（持久运行目录里原地保留）：

    ====================  ================================================
    文件                  内容
    ====================  ================================================
    ``xtbpath.xyz``       路径的逐帧几何（含起点）。注释行 ``energy: X`` 是
                          **相对输入几何的能量，单位 kcal/mol**
    ``xtbpath_ts.xyz``    MTD 找到的 TS **候选**。注释行 ``energy: X`` 是
                          **绝对能量，单位 Eh**
    ``xtbopt.log``        产物侧预优化的轨迹
    ``xtbopt.xyz`` 等     引擎其余中间产物
    ====================  ================================================

    **两个文件的注释行单位不同，这是最容易静默出错的地方**：实测
    ``xtbpath_ts.xyz`` 的 ``-4.987083865354`` 与单点 ``TOTAL ENERGY
-4.987083865355 Eh``
    一致到 1e-12；而 ``xtbpath.xyz`` 里某帧注释 ``52.271`` 对应的真实能量差是
    ``-4.987083865354 - (-5.070383095108) = 0.083299 Eh = 52.272 kcal/mol``。
    因此本模块把两者**分别标注**（``rel_energy_kcal`` / ``ts_candidate_energy_eh``），
    绝不放进同一个字段。

**这个结果只是初猜**：``xtbpath_ts.xyz`` 是 MTD 采到的最高点附近结构，不是鞍点，
更不是"已验证"。验证仍归频率/IRC 判定。

**随机性**：MTD 是随机的，同一次输入重跑路径不同。因此结果按**采样**看待，命令与
stdout 全部落盘以备复查。

**路径精修必须跑到引擎自己判收敛**（本机 xtb 6.7.1 实测，逐字证据与可重跑脚本在
``docs/evidence/ts_guess_mtd_artifact/``）：xTB 的路径是**迭代精修**的，而它的内建上限
只有
3 轮，3 轮后会打印 ``terminated because max. # cycles reached`` —— 那不是收敛，是**
被截断**。
截断路径的最高帧落在反应物自己的盆地里（实测：最高帧离起点只有 21°，相对能 44.865 kcal/mol
，
拿去做 ORCA ``OptTS`` 跑 56 轮不收敛、一路滑回极小值）。把上限放开后，同一个端点对在第 31
轮
```

```
cloudra/tools/core/xtb_path_runner.py:39    由引擎自己判收敛（``terminated because grad at TS <threshold``），barrier 降到 18.299 kcal/mol，
cloudra/tools/core/xtb_path_runner.py:40    最高帧落在扭转极大上，``OptTS`` 9 轮即收敛到一阶鞍点。因此本模块：
cloudra/tools/core/xtb_path_runner.py:41
cloudra/tools/core/xtb_path_runner.py:42    - **总是**写一份 ``$path`` 输入并把 ``nrun`` 上限显式传给引擎（内建默认 3 轮不能用）；
cloudra/tools/core/xtb_path_runner.py:43    - 把这两句终止语**分开分类**：``converged`` / ``cycles_exhausted``；
cloudra/tools/core/xtb_path_runner.py:44    - 触到上限一律报 ``path_not_converged`` 且 ``usable_as_guess=False``，绝不把截断路径当可用初猜。
cloudra/tools/core/xtb_path_runner.py:45    """
cloudra/tools/core/xtb_path_runner.py:46
cloudra/tools/core/xtb_path_runner.py:47    from __future__ import annotations
cloudra/tools/core/xtb_path_runner.py:48
cloudra/tools/core/xtb_path_runner.py:49    import os
cloudra/tools/core/xtb_path_runner.py:50    import subprocess
cloudra/tools/core/xtb_path_runner.py:51    from typing import Literal, Optional
cloudra/tools/core/xtb_path_runner.py:52
cloudra/tools/core/xtb_path_runner.py:53    from pydantic import BaseModel, Field
cloudra/tools/core/xtb_path_runner.py:54
cloudra/tools/core/xtb_path_runner.py:55    from cloudra.persistence.fs_utils import atomic_write_text
cloudra/tools/core/xtb_path_runner.py:56    from cloudra.tools.core.run_dirs import (
cloudra/tools/core/xtb_path_runner.py:57        capture_versions,
cloudra/tools/core/xtb_path_runner.py:58        list_artifact_files,
cloudra/tools/core/xtb_path_runner.py:59        next_run_dir,
cloudra/tools/core/xtb_path_runner.py:60        terminate_process_group,
cloudra/tools/core/xtb_path_runner.py:61    )
cloudra/tools/core/xtb_path_runner.py:62
cloudra/tools/core/xtb_path_runner.py:63    PATH_XYZ_NAME = "xtbpath.xyz"
cloudra/tools/core/xtb_path_runner.py:64    TS_XYZ_NAME = "xtbpath_ts.xyz"
cloudra/tools/core/xtb_path_runner.py:65    RUNS_DIR_NAME = "xtb_path_runs"
cloudra/tools/core/xtb_path_runner.py:66    MAX_EVIDENCE_LINES = 20
cloudra/tools/core/xtb_path_runner.py:67
cloudra/tools/core/xtb_path_runner.py:68    # --- xtb 6.7.1 的逐字标记（精确比对，不做模糊匹配；本模块不含正则） ---
cloudra/tools/core/xtb_path_runner.py:69    MARKER_NORMAL_TERMINATION = "normal termination of xtb"
cloudra/tools/core/xtb_path_runner.py:70    MARKER_FINISHED_RUN = " * finished run on"
cloudra/tools/core/xtb_path_runner.py:71    MARKER_PATH_FINDER = " path finder:"
cloudra/tools/core/xtb_path_runner.py:72    #: 路径精修的两句终止语。前者是引擎判的收敛，后者是**触到轮数上限被截断**，二者绝不等价。
cloudra/tools/core/xtb_path_runner.py:73    MARKER_PATH_CONVERGED = "terminated because grad at TS <threshold"
cloudra/tools/core/xtb_path_runner.py:74    MARKER_PATH_CYCLES_EXHAUSTED = "terminated because max. # cycles reached"
cloudra/tools/core/xtb_path_runner.py:75    #: 每轮精修打印的 barrier（第一列是空格对齐，逐字保留）
cloudra/tools/core/xtb_path_runner.py:76    MARKER_FORWARD_BARRIER = "forward  barrier (kcal)"
cloudra/tools/core/xtb_path_runner.py:77    MARKER_BACKWARD_BARRIER = "backward barrier (kcal)"
cloudra/tools/core/xtb_path_runner.py:78
cloudra/tools/core/xtb_path_runner.py:79    #: 路径精修的轮数上限（``$path nrun``）。这**不是**收敛判据，只是机时上限：收敛由引擎自己判
cloudra/tools/core/xtb_path_runner.py:80    #: （见模块 docstring 的实测）。默认给足，让引擎跑到自己的收敛判据为止。
cloudra/tools/core/xtb_path_runner.py:81    DEFAULT_REFINEMENT_RUNS = 100
cloudra/tools/core/xtb_path_runner.py:82    #: ``$path`` 输入文件名（与命令行一起落盘备查）
cloudra/tools/core/xtb_path_runner.py:83    PATH_INPUT_NAME = "path.inp"
cloudra/tools/core/xtb_path_runner.py:84
cloudra/tools/core/xtb_path_runner.py:85    #: 初猜来源标识（SOP §2：初猜是怎么来的必须写明，不许静默降级）。
cloudra/tools/core/xtb_path_runner.py:86    PATH_KIND_XTB = "xtb_path"
cloudra/tools/core/xtb_path_runner.py:87    PATH_KIND_IDPP = "idpp_fallback"
cloudra/tools/core/xtb_path_runner.py:88
cloudra/tools/core/xtb_path_runner.py:89    PathTermination = Literal[
cloudra/tools/core/xtb_path_runner.py:90        "completed",
```

```
        "executable_missing",
        "timeout",
        "invalid_endpoints",
        "incomplete_artifacts",
        "path_not_converged",
        "failed",
        "unknown_failure",
    ]

    #: 引擎自己报的路径精修结局。``""`` = 日志里没有这两句终止语（本模块按未知处理，不默认成功）。
    RefinementTermination = Literal["", "converged", "cycles_exhausted"]


    class XtbPathRequest(BaseModel):
        """一次 MTD 路径搜索的请求。"""

        reactant_xyz: str = Field(description="反应物几何（单帧 xyz 文本）")
        product_xyz: str = Field(description="产物几何（单帧 xyz 文本）；元素序列必须与反应物一致")

        output_dir: str = Field(description="持久运行目录的父目录；每次运行在其 xtb_path_runs/ 下新建")

        charge: int = Field(default=0, description="总电荷（--chrg）")
        multiplicity: int = Field(default=1, description="自旋多重度 2S+1，透传为 --uhf = multiplicity-1")

        timeout: int = Field(default=900, description="子进程耗时上限（秒）；超时只终止本次运行自己的进程组")

        refinement_runs: int = Field(
            default=DEFAULT_REFINEMENT_RUNS, ge=1,
            description="路径精修的轮数上限（$path nrun）。**只是机时上限**：收敛由引擎自己判，"
                        "触到这个上限即报 path_not_converged。实测内建默认 3 轮远不够收敛，"
                        "不要把它调小",
        )
        run_label: str = Field(default="xtbpath", description="运行目录前缀")
        extra_args: list[str] = Field(default_factory=list, description="追加到命令行的原生选项")


    class XtbArtifact(BaseModel):
        name: str
        path: str
        size_bytes: int


    class PathFrame(BaseModel):
        """路径上的一帧。"""

        frame_index: int
        rel_energy_kcal: Optional[float] = Field(
            default=None,
            description="文件注释里的能量，单位**已核实为 kcal/mol**，相对输入几何；取不到时为 None",
        )
        positions: list[list[float]]


    class XtbPathResult(BaseModel):
        """路径搜索结果。进程退出与"能不能用来当初猜"分开表达。"""
```

```
cloudra/tools/core/xtb_path_runner.py:143       run_dir: str
cloudra/tools/core/xtb_path_runner.py:144       command: list[str] = Field(default_factory=list)
cloudra/tools/core/xtb_path_runner.py:145       returncode: Optional[int] = None
cloudra/tools/core/xtb_path_runner.py:146       path_kind: Literal["xtb_path"] = PATH_KIND_XTB
cloudra/tools/core/xtb_path_runner.py:147       termination_reason: PathTermination = "unknown_failure"
cloudra/tools/core/xtb_path_runner.py:148       refinement_runs: int = Field(default=0, description="本次传给引擎的路径精修轮数上限")
cloudra/tools/core/xtb_path_runner.py:149       refinement_termination: RefinementTermination = Field(
cloudra/tools/core/xtb_path_runner.py:150           default="",
cloudra/tools/core/xtb_path_runner.py:151           description="引擎自己报的精修结局：converged（引擎判收敛）/ cycles_exhausted（触顶截断）",
cloudra/tools/core/xtb_path_runner.py:152       )
cloudra/tools/core/xtb_path_runner.py:153       usable_as_guess: bool = Field(
cloudra/tools/core/xtb_path_runner.py:154           default=False,
cloudra/tools/core/xtb_path_runner.py:155           description="是否产出了可用的初猜（路径 + TS 候选齐全且几何自洽）。"
cloudra/tools/core/xtb_path_runner.py:156                       "**这不等于已验证**：初猜仍需频率/IRC 判定",
cloudra/tools/core/xtb_path_runner.py:157       )
cloudra/tools/core/xtb_path_runner.py:158       symbols: list[str] = Field(default_factory=list)
cloudra/tools/core/xtb_path_runner.py:159       frames: list[PathFrame] = Field(default_factory=list)
cloudra/tools/core/xtb_path_runner.py:160       ts_candidate_energy_eh: Optional[float] = Field(
cloudra/tools/core/xtb_path_runner.py:161           default=None, description="TS 候选的**绝对**能量（Eh）；与路径帧的相对 kcal/mol 不是一个字段"
cloudra/tools/core/xtb_path_runner.py:162       )
cloudra/tools/core/xtb_path_runner.py:163       ts_candidate_positions: list[list[float]] = Field(default_factory=list)
cloudra/tools/core/xtb_path_runner.py:164       reactant_path: str = ""
cloudra/tools/core/xtb_path_runner.py:165       product_path: str = ""
cloudra/tools/core/xtb_path_runner.py:166       command_path: str = ""
cloudra/tools/core/xtb_path_runner.py:167       path_input_path: str = ""
cloudra/tools/core/xtb_path_runner.py:168       versions_path: str = ""
cloudra/tools/core/xtb_path_runner.py:169       stdout_path: str = ""
cloudra/tools/core/xtb_path_runner.py:170       stderr_path: str = ""
cloudra/tools/core/xtb_path_runner.py:171       path_xyz_path: str = ""
cloudra/tools/core/xtb_path_runner.py:172       ts_xyz_path: str = ""
cloudra/tools/core/xtb_path_runner.py:173       artifacts: list[XtbArtifact] = Field(default_factory=list)
cloudra/tools/core/xtb_path_runner.py:174       evidence: list[str] = Field(default_factory=list, description="命中的逐字标记行（原样保留）")
cloudra/tools/core/xtb_path_runner.py:175       diagnostics: list[str] = Field(default_factory=list)
cloudra/tools/core/xtb_path_runner.py:176       message: str = ""
cloudra/tools/core/xtb_path_runner.py:177
cloudra/tools/core/xtb_path_runner.py:178
cloudra/tools/core/xtb_path_runner.py:179   def build_path_input(refinement_runs: int) -> str:
cloudra/tools/core/xtb_path_runner.py:180       """``$path`` 输入块：把精修轮数上限显式交给引擎，**不使用内建默认 3 轮**。
cloudra/tools/core/xtb_path_runner.py:181
cloudra/tools/core/xtb_path_runner.py:182       本机 6.7.1 实测：``$path`` 块被读取；它认得的键是 ``nrun`` / ``npoint`` / ``kpush`` /
cloudra/tools/core/xtb_path_runner.py:183       ``kpull``，其余（``npoints`` / ``alpha`` / ``step`` / ``time`` / ``seed`` 等）会被逐字拒掉
cloudra/tools/core/xtb_path_runner.py:184       （``the key '...' is not recognized by path``），因此这里只写实测可用的键。
cloudra/tools/core/xtb_path_runner.py:185       """
cloudra/tools/core/xtb_path_runner.py:186       if refinement_runs < 1:
cloudra/tools/core/xtb_path_runner.py:187           raise ValueError(f"refinement_runs must be >= 1, got {refinement_runs}")
cloudra/tools/core/xtb_path_runner.py:188       return f"$path\n   nrun={refinement_runs}\n$end\n"
cloudra/tools/core/xtb_path_runner.py:189
cloudra/tools/core/xtb_path_runner.py:190
cloudra/tools/core/xtb_path_runner.py:191   def build_xtb_path_command(request: XtbPathRequest) -> list[str]:
cloudra/tools/core/xtb_path_runner.py:192       """构造命令行。电荷与自旋显式透传，绝不依赖引擎默认推断。"""
cloudra/tools/core/xtb_path_runner.py:193       if request.multiplicity < 1:
cloudra/tools/core/xtb_path_runner.py:194           raise ValueError(f"multiplicity must be >= 1 (2S+1), got {request.multiplicity}")
```

```
cloudra/tools/core/xtb_path_runner.py:195       return [
cloudra/tools/core/xtb_path_runner.py:196           "xtb", "reactant.xyz", "--path", "product.xyz", "--input",
PATH_INPUT_NAME,
cloudra/tools/core/xtb_path_runner.py:197           "--chrg", str(request.charge), "--uhf", str(request.multiplicity - 1),
cloudra/tools/core/xtb_path_runner.py:198           *request.extra_args,
cloudra/tools/core/xtb_path_runner.py:199       ]
cloudra/tools/core/xtb_path_runner.py:200
cloudra/tools/core/xtb_path_runner.py:201
cloudra/tools/core/xtb_path_runner.py:202   def _symbols_and_positions(xyz_text: str, *, label: str) -> tuple[list[str],
list[list[float]]]:
cloudra/tools/core/xtb_path_runner.py:203       """解析单帧 xyz；不完整即 fail-loud。"""
cloudra/tools/core/xtb_path_runner.py:204       lines = xyz_text.splitlines()
cloudra/tools/core/xtb_path_runner.py:205       if not lines:
cloudra/tools/core/xtb_path_runner.py:206           raise ValueError(f"{label}: empty xyz text")
cloudra/tools/core/xtb_path_runner.py:207       try:
cloudra/tools/core/xtb_path_runner.py:208           count = int(lines[0].strip())
cloudra/tools/core/xtb_path_runner.py:209       except ValueError as exc:
cloudra/tools/core/xtb_path_runner.py:210           raise ValueError(f"{label}: first line must be the atom count, got {
lines[0]!r}") from exc
cloudra/tools/core/xtb_path_runner.py:211       if count < 1:
cloudra/tools/core/xtb_path_runner.py:212           raise ValueError(f"{label}: declares {count} atoms")
cloudra/tools/core/xtb_path_runner.py:213       if len(lines) < 2 + count:
cloudra/tools/core/xtb_path_runner.py:214           raise ValueError(f"{label}: declares {count} atoms but has too few
lines")
cloudra/tools/core/xtb_path_runner.py:215       symbols: list[str] = []
cloudra/tools/core/xtb_path_runner.py:216       positions: list[list[float]] = []
cloudra/tools/core/xtb_path_runner.py:217       for line in lines[2: 2 + count]:
cloudra/tools/core/xtb_path_runner.py:218           parts = line.split()
cloudra/tools/core/xtb_path_runner.py:219           if len(parts) < 4:
cloudra/tools/core/xtb_path_runner.py:220               raise ValueError(f"{label}: malformed coordinate line {line!r}")
cloudra/tools/core/xtb_path_runner.py:221           symbols.append(parts[0])
cloudra/tools/core/xtb_path_runner.py:222           positions.append([float(parts[1]), float(parts[2]), float(parts[3])])
cloudra/tools/core/xtb_path_runner.py:223       return symbols, positions
cloudra/tools/core/xtb_path_runner.py:224
cloudra/tools/core/xtb_path_runner.py:225
cloudra/tools/core/xtb_path_runner.py:226   def _parse_path_frames(path: str) -> list[PathFrame]:
cloudra/tools/core/xtb_path_runner.py:227       """解析 ``xtbpath.xyz``。注释里的能量是**相对 kcal/mol**，取不到就留 None，不猜。
"""
cloudra/tools/core/xtb_path_runner.py:228       with open(path, encoding="utf-8", errors="replace") as handle:
cloudra/tools/core/xtb_path_runner.py:229           lines = handle.read().splitlines()
cloudra/tools/core/xtb_path_runner.py:230       frames: list[PathFrame] = []
cloudra/tools/core/xtb_path_runner.py:231       index = 0
cloudra/tools/core/xtb_path_runner.py:232       while index < len(lines):
cloudra/tools/core/xtb_path_runner.py:233           if not lines[index].strip():
cloudra/tools/core/xtb_path_runner.py:234               index += 1
cloudra/tools/core/xtb_path_runner.py:235               continue
cloudra/tools/core/xtb_path_runner.py:236           try:
cloudra/tools/core/xtb_path_runner.py:237               count = int(lines[index].strip())
cloudra/tools/core/xtb_path_runner.py:238           except ValueError:
cloudra/tools/core/xtb_path_runner.py:239               break
cloudra/tools/core/xtb_path_runner.py:240           comment = lines[index + 1] if index + 1 < len(lines) else ""
cloudra/tools/core/xtb_path_runner.py:241           rel: Optional[float] = None
cloudra/tools/core/xtb_path_runner.py:242           tokens = comment.split()
cloudra/tools/core/xtb_path_runner.py:243           # 形如 ``energy: <float> xtb: <version>``；逐 token 取，不做模糊匹配
cloudra/tools/core/xtb_path_runner.py:244           if len(tokens) >= 2 and tokens[0] == "energy:":
cloudra/tools/core/xtb_path_runner.py:245               try:
cloudra/tools/core/xtb_path_runner.py:246                   rel = float(tokens[1])
```

```
cloudra/tools/core/xtb_path_runner.py:247             except ValueError:
cloudra/tools/core/xtb_path_runner.py:248                 rel = None
cloudra/tools/core/xtb_path_runner.py:249         block = lines[index + 2: index + 2 + count]
cloudra/tools/core/xtb_path_runner.py:250         if len(block) != count:
cloudra/tools/core/xtb_path_runner.py:251             break
cloudra/tools/core/xtb_path_runner.py:252         positions = []
cloudra/tools/core/xtb_path_runner.py:253         for line in block:
cloudra/tools/core/xtb_path_runner.py:254             parts = line.split()
cloudra/tools/core/xtb_path_runner.py:255             if len(parts) < 4:
cloudra/tools/core/xtb_path_runner.py:256                 raise ValueError(f"{path}: malformed coordinate line {line!r}")
cloudra/tools/core/xtb_path_runner.py:257             positions.append([float(parts[1]), float(parts[2]), float(parts[3])
])
cloudra/tools/core/xtb_path_runner.py:258         frames.append(PathFrame(frame_index=len(frames), rel_energy_kcal=rel,
positions=positions))
cloudra/tools/core/xtb_path_runner.py:259         index += 2 + count
cloudra/tools/core/xtb_path_runner.py:260     if not frames:
cloudra/tools/core/xtb_path_runner.py:261         raise ValueError(f"{path}: no complete frame could be read")
cloudra/tools/core/xtb_path_runner.py:262     return frames
cloudra/tools/core/xtb_path_runner.py:263
cloudra/tools/core/xtb_path_runner.py:264
cloudra/tools/core/xtb_path_runner.py:265 def _parse_ts_candidate(path: str) -> tuple[float, list[list[float]]]:
cloudra/tools/core/xtb_path_runner.py:266     """解析 ``xtbpath_ts.xyz``。注释里的能量是**绝对 Eh**——与路径帧的单位不同。"""
cloudra/tools/core/xtb_path_runner.py:267     with open(path, encoding="utf-8", errors="replace") as handle:
cloudra/tools/core/xtb_path_runner.py:268         lines = handle.read().splitlines()
cloudra/tools/core/xtb_path_runner.py:269     if len(lines) < 3:
cloudra/tools/core/xtb_path_runner.py:270         raise ValueError(f"{path}: too few lines for a single frame")
cloudra/tools/core/xtb_path_runner.py:271     count = int(lines[0].strip())
cloudra/tools/core/xtb_path_runner.py:272     tokens = lines[1].split()
cloudra/tools/core/xtb_path_runner.py:273     if len(tokens) < 2 or tokens[0] != "energy:":
cloudra/tools/core/xtb_path_runner.py:274         raise ValueError(
cloudra/tools/core/xtb_path_runner.py:275             f"{path}: comment line must start with 'energy: <Eh>', got {lines
[1]!r}"
cloudra/tools/core/xtb_path_runner.py:276         )
cloudra/tools/core/xtb_path_runner.py:277     energy = float(tokens[1])
cloudra/tools/core/xtb_path_runner.py:278     block = lines[2: 2 + count]
cloudra/tools/core/xtb_path_runner.py:279     if len(block) != count:
cloudra/tools/core/xtb_path_runner.py:280         raise ValueError(f"{path}: declares {count} atoms but only {len(block)}
 coordinates follow")
cloudra/tools/core/xtb_path_runner.py:281     positions = []
cloudra/tools/core/xtb_path_runner.py:282     for line in block:
cloudra/tools/core/xtb_path_runner.py:283         parts = line.split()
cloudra/tools/core/xtb_path_runner.py:284         if len(parts) < 4:
cloudra/tools/core/xtb_path_runner.py:285             raise ValueError(f"{path}: malformed coordinate line {line!r}")
cloudra/tools/core/xtb_path_runner.py:286         positions.append([float(parts[1]), float(parts[2]), float(parts[3])])
cloudra/tools/core/xtb_path_runner.py:287     return energy, positions
cloudra/tools/core/xtb_path_runner.py:288
cloudra/tools/core/xtb_path_runner.py:289
cloudra/tools/core/xtb_path_runner.py:290 class PathMarkers(BaseModel):
cloudra/tools/core/xtb_path_runner.py:291     """日志里命中的逐字标记。"""
cloudra/tools/core/xtb_path_runner.py:292
cloudra/tools/core/xtb_path_runner.py:293     evidence: list[str] = Field(default_factory=list)
cloudra/tools/core/xtb_path_runner.py:294     normal_termination: bool = False
cloudra/tools/core/xtb_path_runner.py:295     path_finder: bool = False
cloudra/tools/core/xtb_path_runner.py:296     refinement_termination: RefinementTermination = ""
cloudra/tools/core/xtb_path_runner.py:297
cloudra/tools/core/xtb_path_runner.py:298
```

```
def _scan_markers(lines: list[str]) -> PathMarkers:
    """精确 token 扫描。两条终止语**分开**判定：收敛与触顶绝不等价。

    barrier 行每轮精修都打印一次，只保留**最后一次**（更靠后的那个才是当前精修程度）。
    """
    markers = PathMarkers()
    last_forward = ""
    last_backward = ""
    for line in lines:
        stripped = line.strip()
        if MARKER_NORMAL_TERMINATION in stripped:
            markers.evidence.append(stripped)
            markers.normal_termination = True
        elif stripped.startswith(MARKER_FINISHED_RUN.strip()):
            markers.evidence.append(stripped)
        elif MARKER_PATH_FINDER.strip() in stripped:
            markers.evidence.append(stripped)
            markers.path_finder = True
        elif MARKER_PATH_CONVERGED in stripped:
            markers.evidence.append(stripped)
            markers.refinement_termination = "converged"
        elif MARKER_PATH_CYCLES_EXHAUSTED in stripped:
            markers.evidence.append(stripped)
            markers.refinement_termination = "cycles_exhausted"
        elif MARKER_FORWARD_BARRIER in stripped:
            last_forward = stripped
        elif MARKER_BACKWARD_BARRIER in stripped:
            last_backward = stripped
    for extra in (last_forward, last_backward):
        if extra:
            markers.evidence.append(extra)
    markers.evidence = markers.evidence[:MAX_EVIDENCE_LINES]
    return markers


def run_xtb_path(request: XtbPathRequest) -> XtbPathResult:
    """在持久运行目录里跑一次 MTD 路径搜索。

    同步函数（子进程 + 文件 IO）；异步调用方必须经 ``asyncio.to_thread`` 卸载。
    """
    try:
        reactant_symbols, _ = _symbols_and_positions(request.reactant_xyz, label="reactant")
        product_symbols, _ = _symbols_and_positions(request.product_xyz, label="product")
    except ValueError as exc:
        # 端点先自检：几何不完整或元素序列不一致时，跑引擎只会浪费机时
        output_dir = request.output_dir
        os.makedirs(output_dir, exist_ok=True)
        run_dir = next_run_dir(output_dir, RUNS_DIR_NAME, request.run_label)
        return XtbPathResult(
            run_dir=run_dir, termination_reason="invalid_endpoints",
            message=f"the endpoints were rejected before starting xtb: {exc}",
            diagnostics=[str(exc)],
```

```
cloudra/tools/core/xtb_path_runner.py:351           )
cloudra/tools/core/xtb_path_runner.py:352
cloudra/tools/core/xtb_path_runner.py:353       command = build_xtb_path_command(request)
cloudra/tools/core/xtb_path_runner.py:354       run_dir = next_run_dir(request.output_dir, RUNS_DIR_NAME, request.run_label
)
cloudra/tools/core/xtb_path_runner.py:355       reactant_path = os.path.join(run_dir, "reactant.xyz")
cloudra/tools/core/xtb_path_runner.py:356       product_path = os.path.join(run_dir, "product.xyz")
cloudra/tools/core/xtb_path_runner.py:357       command_path = os.path.join(run_dir, "command.txt")
cloudra/tools/core/xtb_path_runner.py:358       path_input_path = os.path.join(run_dir, PATH_INPUT_NAME)
cloudra/tools/core/xtb_path_runner.py:359       versions_path = os.path.join(run_dir, "versions.txt")
cloudra/tools/core/xtb_path_runner.py:360       stdout_path = os.path.join(run_dir, "stdout.log")
cloudra/tools/core/xtb_path_runner.py:361       stderr_path = os.path.join(run_dir, "stderr.log")
cloudra/tools/core/xtb_path_runner.py:362
cloudra/tools/core/xtb_path_runner.py:363       atomic_write_text(reactant_path, request.reactant_xyz)
cloudra/tools/core/xtb_path_runner.py:364       atomic_write_text(product_path, request.product_xyz)
cloudra/tools/core/xtb_path_runner.py:365       atomic_write_text(command_path, " ".join(command) + "\n")
cloudra/tools/core/xtb_path_runner.py:366       atomic_write_text(path_input_path, build_path_input(request.refinement_runs
))
cloudra/tools/core/xtb_path_runner.py:367       atomic_write_text(versions_path, capture_versions(("xtb",), ("xtb version
",)))
cloudra/tools/core/xtb_path_runner.py:368
cloudra/tools/core/xtb_path_runner.py:369       result = XtbPathResult(
cloudra/tools/core/xtb_path_runner.py:370           run_dir=run_dir, command=command, symbols=reactant_symbols,
cloudra/tools/core/xtb_path_runner.py:371           reactant_path=reactant_path, product_path=product_path,
cloudra/tools/core/xtb_path_runner.py:372           command_path=command_path, path_input_path=path_input_path,
versions_path=versions_path,
cloudra/tools/core/xtb_path_runner.py:373           refinement_runs=request.refinement_runs,
cloudra/tools/core/xtb_path_runner.py:374           stdout_path=stdout_path, stderr_path=stderr_path,
cloudra/tools/core/xtb_path_runner.py:375           path_xyz_path=os.path.join(run_dir, PATH_XYZ_NAME),
cloudra/tools/core/xtb_path_runner.py:376           ts_xyz_path=os.path.join(run_dir, TS_XYZ_NAME),
cloudra/tools/core/xtb_path_runner.py:377       )
cloudra/tools/core/xtb_path_runner.py:378
cloudra/tools/core/xtb_path_runner.py:379       if reactant_symbols != product_symbols:
cloudra/tools/core/xtb_path_runner.py:380           result.termination_reason = "invalid_endpoints"
cloudra/tools/core/xtb_path_runner.py:381           result.diagnostics.append(
cloudra/tools/core/xtb_path_runner.py:382               f"reactant and product element sequences differ ({reactant_symbols}
 vs "
cloudra/tools/core/xtb_path_runner.py:383               f"{product_symbols}); the path search needs one consistent atom set
"
cloudra/tools/core/xtb_path_runner.py:384           )
cloudra/tools/core/xtb_path_runner.py:385           result.message = "Endpoints rejected: the element sequences differ."
cloudra/tools/core/xtb_path_runner.py:386           result.artifacts = [
cloudra/tools/core/xtb_path_runner.py:387               XtbArtifact(name=name, path=path, size_bytes=size)
cloudra/tools/core/xtb_path_runner.py:388               for name, path, size in list_artifact_files(run_dir)
cloudra/tools/core/xtb_path_runner.py:389           ]
cloudra/tools/core/xtb_path_runner.py:390           return result
cloudra/tools/core/xtb_path_runner.py:391
cloudra/tools/core/xtb_path_runner.py:392       import shutil
cloudra/tools/core/xtb_path_runner.py:393
cloudra/tools/core/xtb_path_runner.py:394       if not shutil.which("xtb"):
cloudra/tools/core/xtb_path_runner.py:395           result.termination_reason = "executable_missing"
cloudra/tools/core/xtb_path_runner.py:396           result.diagnostics.append(
cloudra/tools/core/xtb_path_runner.py:397               "the 'xtb' executable was not found on PATH; no run was started.
The caller should "
cloudra/tools/core/xtb_path_runner.py:398               f"fall back to IDPP interpolation and report path_kind={
PATH_KIND_IDPP!r}"
cloudra/tools/core/xtb_path_runner.py:399           )
cloudra/tools/core/xtb_path_runner.py:400           atomic_write_text(stdout_path, "")
cloudra/tools/core/xtb_path_runner.py:401           atomic_write_text(stderr_path, "")
cloudra/tools/core/xtb_path_runner.py:402           result.artifacts = [
```

```
cloudra/tools/core/xtb_path_runner.py:403                XtbArtifact(name=name, path=path, size_bytes=size)
cloudra/tools/core/xtb_path_runner.py:404                for name, path, size in list_artifact_files(run_dir)
cloudra/tools/core/xtb_path_runner.py:405            ]
cloudra/tools/core/xtb_path_runner.py:406            result.message = "xtb executable not found; no path search was started
."
cloudra/tools/core/xtb_path_runner.py:407            return result
cloudra/tools/core/xtb_path_runner.py:408
cloudra/tools/core/xtb_path_runner.py:409        timed_out = False
cloudra/tools/core/xtb_path_runner.py:410        proc = subprocess.Popen(
cloudra/tools/core/xtb_path_runner.py:411            command, cwd=run_dir, stdout=subprocess.PIPE, stderr=subprocess.PIPE,
cloudra/tools/core/xtb_path_runner.py:412            text=True, start_new_session=True,
cloudra/tools/core/xtb_path_runner.py:413        )
cloudra/tools/core/xtb_path_runner.py:414        try:
cloudra/tools/core/xtb_path_runner.py:415            stdout, stderr = proc.communicate(timeout=request.timeout)
cloudra/tools/core/xtb_path_runner.py:416            returncode = proc.returncode
cloudra/tools/core/xtb_path_runner.py:417        except subprocess.TimeoutExpired:
cloudra/tools/core/xtb_path_runner.py:418            timed_out = True
cloudra/tools/core/xtb_path_runner.py:419            terminate_process_group(proc)
cloudra/tools/core/xtb_path_runner.py:420            stdout, stderr = proc.communicate()
cloudra/tools/core/xtb_path_runner.py:421            returncode = proc.returncode
cloudra/tools/core/xtb_path_runner.py:422            result.diagnostics.append(
cloudra/tools/core/xtb_path_runner.py:423                f"the xtb path search exceeded the {request.timeout}s timeout; this
 run's own process "
cloudra/tools/core/xtb_path_runner.py:424                "group was terminated (its logs and intermediate files are kept)"
cloudra/tools/core/xtb_path_runner.py:425            )
cloudra/tools/core/xtb_path_runner.py:426
cloudra/tools/core/xtb_path_runner.py:427        atomic_write_text(stdout_path, stdout or "")
cloudra/tools/core/xtb_path_runner.py:428        atomic_write_text(stderr_path, stderr or "")
cloudra/tools/core/xtb_path_runner.py:429        result.returncode = returncode
cloudra/tools/core/xtb_path_runner.py:430        result.artifacts = [
cloudra/tools/core/xtb_path_runner.py:431            XtbArtifact(name=name, path=path, size_bytes=size)
cloudra/tools/core/xtb_path_runner.py:432            for name, path, size in list_artifact_files(run_dir)
cloudra/tools/core/xtb_path_runner.py:433        ]
cloudra/tools/core/xtb_path_runner.py:434
cloudra/tools/core/xtb_path_runner.py:435        # 实测（本机 xtb 6.7.1）：``normal termination of xtb`` 落在 **stderr**，而计时
块落在
cloudra/tools/core/xtb_path_runner.py:436        # stdout。因此标记必须在两个流上一起扫——只看 stdout 会把正常的运行判成失败。
cloudra/tools/core/xtb_path_runner.py:437        markers = _scan_markers(
cloudra/tools/core/xtb_path_runner.py:438            (stdout or "").splitlines() + (stderr or "").splitlines()
cloudra/tools/core/xtb_path_runner.py:439        )
cloudra/tools/core/xtb_path_runner.py:440        result.evidence = markers.evidence
cloudra/tools/core/xtb_path_runner.py:441        result.refinement_termination = markers.refinement_termination
cloudra/tools/core/xtb_path_runner.py:442
cloudra/tools/core/xtb_path_runner.py:443        if timed_out:
cloudra/tools/core/xtb_path_runner.py:444            result.termination_reason = "timeout"
cloudra/tools/core/xtb_path_runner.py:445            result.message = (
cloudra/tools/core/xtb_path_runner.py:446                f"the xtb path search timed out after {request.timeout}s (
returncode={returncode}). "
cloudra/tools/core/xtb_path_runner.py:447                f"Full log: {stdout_path}"
cloudra/tools/core/xtb_path_runner.py:448            )
cloudra/tools/core/xtb_path_runner.py:449            return result
cloudra/tools/core/xtb_path_runner.py:450
cloudra/tools/core/xtb_path_runner.py:451        if returncode != 0:
cloudra/tools/core/xtb_path_runner.py:452            result.termination_reason = "failed"
cloudra/tools/core/xtb_path_runner.py:453            tail = "\n".join((stdout or "").splitlines()[-15:]) or (stderr or "")
[-2000:]
cloudra/tools/core/xtb_path_runner.py:454            result.diagnostics.append(
```

```
cloudra/tools/core/xtb_path_runner.py:455                "xtb exited non-zero; raw log tail follows (no cause is guessed)"
cloudra/tools/core/xtb_path_runner.py:456            )
cloudra/tools/core/xtb_path_runner.py:457            result.diagnostics.append(tail)
cloudra/tools/core/xtb_path_runner.py:458            result.message = (
cloudra/tools/core/xtb_path_runner.py:459                f"the xtb path search failed (returncode={returncode}). Full log: {
stdout_path}"
cloudra/tools/core/xtb_path_runner.py:460            )
cloudra/tools/core/xtb_path_runner.py:461            return result
cloudra/tools/core/xtb_path_runner.py:462
cloudra/tools/core/xtb_path_runner.py:463        if not markers.normal_termination:
cloudra/tools/core/xtb_path_runner.py:464            result.termination_reason = "unknown_failure"
cloudra/tools/core/xtb_path_runner.py:465            result.diagnostics.append(
cloudra/tools/core/xtb_path_runner.py:466                f"xtb exited 0 but never printed its normal-termination marker "
cloudra/tools/core/xtb_path_runner.py:467                f"({MARKER_NORMAL_TERMINATION!r}, which this build writes to stderr
); refusing to treat "
cloudra/tools/core/xtb_path_runner.py:468                "the exit code alone as success"
cloudra/tools/core/xtb_path_runner.py:469            )
cloudra/tools/core/xtb_path_runner.py:470            result.message = f"xtb did not report normal termination. Full log: {
stdout_path}"
cloudra/tools/core/xtb_path_runner.py:471            return result
cloudra/tools/core/xtb_path_runner.py:472
cloudra/tools/core/xtb_path_runner.py:473        if not markers.path_finder:
cloudra/tools/core/xtb_path_runner.py:474            result.termination_reason = "unknown_failure"
cloudra/tools/core/xtb_path_runner.py:475            result.diagnostics.append(
cloudra/tools/core/xtb_path_runner.py:476                f"xtb exited 0 but never printed its path-finder timing block "
cloudra/tools/core/xtb_path_runner.py:477                f"({MARKER_PATH_FINDER.strip()!r}); refusing to treat the exit code
 alone as success"
cloudra/tools/core/xtb_path_runner.py:478            )
cloudra/tools/core/xtb_path_runner.py:479            result.message = f"xtb did not report a completed path search. Full log
: {stdout_path}"
cloudra/tools/core/xtb_path_runner.py:480            return result
cloudra/tools/core/xtb_path_runner.py:481
cloudra/tools/core/xtb_path_runner.py:482        # 产物完整性：路径与 TS 候选都要在，且几何自洽——"文件在"不构成成功判据
cloudra/tools/core/xtb_path_runner.py:483        try:
cloudra/tools/core/xtb_path_runner.py:484            frames = _parse_path_frames(result.path_xyz_path)
cloudra/tools/core/xtb_path_runner.py:485            ts_energy, ts_positions = _parse_ts_candidate(result.ts_xyz_path)
cloudra/tools/core/xtb_path_runner.py:486        except (OSError, ValueError) as exc:
cloudra/tools/core/xtb_path_runner.py:487            result.termination_reason = "incomplete_artifacts"
cloudra/tools/core/xtb_path_runner.py:488            result.diagnostics.append(f"path artifacts are unusable: {exc}")
cloudra/tools/core/xtb_path_runner.py:489            result.message = (
cloudra/tools/core/xtb_path_runner.py:490                f"xtb reported normal termination but its path artifacts are
unusable ({exc}). "
cloudra/tools/core/xtb_path_runner.py:491                f"Full log: {stdout_path}"
cloudra/tools/core/xtb_path_runner.py:492            )
cloudra/tools/core/xtb_path_runner.py:493            return result
cloudra/tools/core/xtb_path_runner.py:494
cloudra/tools/core/xtb_path_runner.py:495        for frame in frames:
cloudra/tools/core/xtb_path_runner.py:496            if len(frame.positions) != len(reactant_symbols):
cloudra/tools/core/xtb_path_runner.py:497                result.termination_reason = "incomplete_artifacts"
cloudra/tools/core/xtb_path_runner.py:498                result.diagnostics.append(
cloudra/tools/core/xtb_path_runner.py:499                    f"frame {frame.frame_index} has {len(frame.positions)} atoms
but the endpoints have "
cloudra/tools/core/xtb_path_runner.py:500                    f"{len(reactant_symbols)}"
cloudra/tools/core/xtb_path_runner.py:501                )
cloudra/tools/core/xtb_path_runner.py:502                result.message = "xtb reported normal termination but the path
frames are inconsistent."
cloudra/tools/core/xtb_path_runner.py:503                return result
cloudra/tools/core/xtb_path_runner.py:504        if len(frames) < 2:
cloudra/tools/core/xtb_path_runner.py:505            result.termination_reason = "incomplete_artifacts"
cloudra/tools/core/xtb_path_runner.py:506            result.diagnostics.append(
```

```
cloudra/tools/core/xtb_path_runner.py:507                f"the path has only {len(frames)} frame(s); a path needs at least the two endpoints"
cloudra/tools/core/xtb_path_runner.py:508            )
cloudra/tools/core/xtb_path_runner.py:509            result.message = "xtb reported normal termination but produced no usable path."
cloudra/tools/core/xtb_path_runner.py:510            return result
cloudra/tools/core/xtb_path_runner.py:511        if len(ts_positions) != len(reactant_symbols):
cloudra/tools/core/xtb_path_runner.py:512            result.termination_reason = "incomplete_artifacts"
cloudra/tools/core/xtb_path_runner.py:513            result.diagnostics.append(
cloudra/tools/core/xtb_path_runner.py:514                f"the TS candidate has {len(ts_positions)} atoms but the endpoints have "
cloudra/tools/core/xtb_path_runner.py:515                f"{len(reactant_symbols)}"
cloudra/tools/core/xtb_path_runner.py:516            )
cloudra/tools/core/xtb_path_runner.py:517            result.message = "xtb reported normal termination but the TS candidate is inconsistent."
cloudra/tools/core/xtb_path_runner.py:518            return result
cloudra/tools/core/xtb_path_runner.py:519
cloudra/tools/core/xtb_path_runner.py:520        result.frames = frames
cloudra/tools/core/xtb_path_runner.py:521        result.ts_candidate_energy_eh = ts_energy
cloudra/tools/core/xtb_path_runner.py:522        result.ts_candidate_positions = ts_positions
cloudra/tools/core/xtb_path_runner.py:523
cloudra/tools/core/xtb_path_runner.py:524        if markers.refinement_termination == "cycles_exhausted":
cloudra/tools/core/xtb_path_runner.py:525            # **被截断的路径不是初猜**：实测这种路径的最高帧落在反应物自己的盆地里
cloudra/tools/core/xtb_path_runner.py:526            # （离起点只有 21°、相对能 44.865 kcal/mol，做 OptTS 跑 56 轮不收敛）。
cloudra/tools/core/xtb_path_runner.py:527            # 这里把截断的现场（路径帧 + TS 候选）保留下来供复查，但明确标成不可用。
cloudra/tools/core/xtb_path_runner.py:528            result.termination_reason = "path_not_converged"
cloudra/tools/core/xtb_path_runner.py:529            result.usable_as_guess = False
cloudra/tools/core/xtb_path_runner.py:530            result.diagnostics.append(
cloudra/tools/core/xtb_path_runner.py:531                f"the engine stopped its path refinement at the {request.refinement_runs}-run cap "
cloudra/tools/core/xtb_path_runner.py:532                f"({MARKER_PATH_CYCLES_EXHAUSTED!r}); that is a truncated path, not a converged one. "
cloudra/tools/core/xtb_path_runner.py:533                "The frames and TS candidate are kept for inspection only. Measured consequence of "
cloudra/tools/core/xtb_path_runner.py:534                "accepting a truncated path (docs/evidence/ts_guess_mtd_artifact/): its highest frame "
cloudra/tools/core/xtb_path_runner.py:535                "sits in the reactant's own basin, 21 deg from the start, and an ORCA OptTS started "
cloudra/tools/core/xtb_path_runner.py:536                "from it never converged (56 cycles, slid back to the minimum). Raise "
cloudra/tools/core/xtb_path_runner.py:537                "`refinement_runs` and rerun"
cloudra/tools/core/xtb_path_runner.py:538            )
cloudra/tools/core/xtb_path_runner.py:539            result.message = (
cloudra/tools/core/xtb_path_runner.py:540                f"the xTB path refinement hit its {request.refinement_runs}-run cap before the engine "
cloudra/tools/core/xtb_path_runner.py:541                f"declared convergence, so no usable TS guess was produced. Raise `refinement_runs` and "
cloudra/tools/core/xtb_path_runner.py:542                f"rerun. Run directory (truncated path kept for inspection): {run_dir}"
cloudra/tools/core/xtb_path_runner.py:543            )
cloudra/tools/core/xtb_path_runner.py:544            return result
cloudra/tools/core/xtb_path_runner.py:545
cloudra/tools/core/xtb_path_runner.py:546        if markers.refinement_termination != "converged":
cloudra/tools/core/xtb_path_runner.py:547            result.termination_reason = "unknown_failure"
cloudra/tools/core/xtb_path_runner.py:548            result.usable_as_guess = False
cloudra/tools/core/xtb_path_runner.py:549            result.diagnostics.append(
cloudra/tools/core/xtb_path_runner.py:550                "xtb exited 0 and produced path artifacts, but its log carries neither refinement "
cloudra/tools/core/xtb_path_runner.py:551                f"termination line ({MARKER_PATH_CONVERGED!r} / {MARKER_PATH_CYCLES_EXHAUSTED!r}); "
cloudra/tools/core/xtb_path_runner.py:552                "whether the path refinement converged cannot be established from this output, and a "
cloudra/tools/core/xtb_path_runner.py:553                "non-converged path must not be handed over as a TS guess"
cloudra/tools/core/xtb_path_runner.py:554            )
cloudra/tools/core/xtb_path_runner.py:555            result.message = (
cloudra/tools/core/xtb_path_runner.py:556                f"the xTB path search reported no path-refinement termination line, so convergence "
cloudra/tools/core/xtb_path_runner.py:557                f"could not be confirmed. Full log: {stdout_path}"
cloudra/tools/core/xtb_path_runner.py:558            )
```

```
cloudra/tools/core/xtb_path_runner.py:559           return result
cloudra/tools/core/xtb_path_runner.py:560
cloudra/tools/core/xtb_path_runner.py:561       result.termination_reason = "completed"
cloudra/tools/core/xtb_path_runner.py:562       result.usable_as_guess = True
cloudra/tools/core/xtb_path_runner.py:563       result.diagnostics.append(
cloudra/tools/core/xtb_path_runner.py:564           "path-frame comment energies are RELATIVE kcal/mol; the TS candidate's comment energy is "
cloudra/tools/core/xtb_path_runner.py:565           "ABSOLUTE Eh — the two are reported in separate fields on purpose "
cloudra/tools/core/xtb_path_runner.py:566           "(verified against single points on this machine)"
cloudra/tools/core/xtb_path_runner.py:567       )
cloudra/tools/core/xtb_path_runner.py:568       result.diagnostics.append(
cloudra/tools/core/xtb_path_runner.py:569           "the TS candidate is a guess sampled by metadynamics, NOT a saddle point and NOT verified; "
cloudra/tools/core/xtb_path_runner.py:570           "frequency/IRC judgement still has to run"
cloudra/tools/core/xtb_path_runner.py:571       )
cloudra/tools/core/xtb_path_runner.py:572       result.diagnostics.append(
cloudra/tools/core/xtb_path_runner.py:573           f"the engine declared the path refinement converged ({MARKER_PATH_CONVERGED!r}) within the "
cloudra/tools/core/xtb_path_runner.py:574           f"{request.refinement_runs}-run cap"
cloudra/tools/core/xtb_path_runner.py:575       )
cloudra/tools/core/xtb_path_runner.py:576       result.message = (
cloudra/tools/core/xtb_path_runner.py:577           f"xtb path search completed: {len(frames)} path frames and a TS candidate "
cloudra/tools/core/xtb_path_runner.py:578           f"({ts_energy:.6f} Eh). Run directory: {run_dir}"
cloudra/tools/core/xtb_path_runner.py:579       )
cloudra/tools/core/xtb_path_runner.py:580       return result
cloudra/tools/name_hints.py:1   """Retired wire names → hint only. Never executed."""
cloudra/tools/name_hints.py:2
cloudra/tools/name_hints.py:3   _RETIRED: dict[str, str] = {
cloudra/tools/name_hints.py:4       "text_reader": "read",
cloudra/tools/name_hints.py:5       "image_reader": "read",
cloudra/tools/name_hints.py:6       "pdf2md": "read",
cloudra/tools/name_hints.py:7       "pdf2image": "read",
cloudra/tools/name_hints.py:8       "read_file": "read",
cloudra/tools/name_hints.py:9       "web_search": "search (source=web)",
cloudra/tools/name_hints.py:10      "paper_searcher": "search (source=arxiv)",
cloudra/tools/name_hints.py:11      "openalex_searcher": "search (source=openalex)",
cloudra/tools/name_hints.py:12      "bashtools": "bash",
cloudra/tools/name_hints.py:13      "shell": "bash",
cloudra/tools/name_hints.py:14      "text_writer": "write",
cloudra/tools/name_hints.py:15      "write_file": "write",
cloudra/tools/name_hints.py:16      "edit_file": "edit",
cloudra/tools/name_hints.py:17      "str_replace": "edit",
cloudra/tools/name_hints.py:18      "getskill": "get_skill",
cloudra/tools/name_hints.py:19      "web_fetcher": "browser",
cloudra/tools/name_hints.py:20      "web_fetch": "browser",
cloudra/tools/name_hints.py:21      "headless_browser": "browser",
cloudra/tools/name_hints.py:22      "SubAgentLauncher": "subagent_launcher",
cloudra/tools/name_hints.py:23      "SubAgentSwarm": "subagent_swarm",
cloudra/tools/name_hints.py:24  }
cloudra/tools/name_hints.py:25
cloudra/tools/name_hints.py:26
cloudra/tools/name_hints.py:27  def hint_unknown_tool(name: str) -> str:
cloudra/tools/name_hints.py:28      target = _RETIRED.get(name)
cloudra/tools/name_hints.py:29      if target:
cloudra/tools/name_hints.py:30          return (
```

```
cloudra/tools/name_hints.py:31              f"[Tool error] unknown tool {name!r}. "
cloudra/tools/name_hints.py:32              f"Use `{target}` instead."
cloudra/tools/name_hints.py:33          )
cloudra/tools/name_hints.py:34      return (
cloudra/tools/name_hints.py:35          f"[Tool error] unknown tool {name!r}. "
cloudra/tools/name_hints.py:36          "Choose an available tool and retry."
cloudra/tools/name_hints.py:37      )
cloudra/tools/name_hints.py:38
cloudra/tools/name_hints.py:39
cloudra/tools/name_hints.py:40  def is_retired_tool_name(name: str) -> bool:
cloudra/tools/name_hints.py:41      return name in _RETIRED
cloudra/tools/registry.py:1 """Explicit built-in tool manifest.
cloudra/tools/registry.py:2
cloudra/tools/registry.py:3 Keep this module dependency-free: :mod:`cloudra.core.agent` imports
cloudra/tools/registry.py:4 ``cloudra.tools`` while several tool implementations import ``cloudra.core.agent``.
cloudra/tools/registry.py:5 The registry therefore stores import paths instead of importing tool classes at
cloudra/tools/registry.py:6 module import time.
cloudra/tools/registry.py:7
cloudra/tools/registry.py:8 Optional semantic fields are the single source of truth for downstream policy
cloudra/tools/registry.py:9 projections (DAG gate / runtime profiles / evidence derivation).  Downstream
cloudra/tools/registry.py:10    modules must query :mod:`cloudra.tools.capabilities` instead of maintaining
cloudra/tools/registry.py:11    their own per-tool name sets.
cloudra/tools/registry.py:12    """
cloudra/tools/registry.py:13
cloudra/tools/registry.py:14    from dataclasses import dataclass
cloudra/tools/registry.py:15
cloudra/tools/registry.py:16
cloudra/tools/registry.py:17    @dataclass(frozen=True)
cloudra/tools/registry.py:18    class ToolRegistration:
cloudra/tools/registry.py:19        module: str
cloudra/tools/registry.py:20        class_name: str
cloudra/tools/registry.py:21
cloudra/tools/registry.py:22        # Declared wire name. Only needed when the tool has non-default semantics
cloudra/tools/registry.py:23        # (or a tool_name that differs from class_name); capabilities only projects
cloudra/tools/registry.py:24        # entries that carry it.
cloudra/tools/registry.py:25        tool_name: str = ""
cloudra/tools/registry.py:26
cloudra/tools/registry.py:27        # DAG semantics. All flags default to the conservative/plain behavior.
cloudra/tools/registry.py:28        dag_observe: bool = True
cloudra/tools/registry.py:29        evidence_deriver: str = ""  # "module:function(current, role, arguments, result)->dict"
cloudra/tools/registry.py:30        # 事实适配器（contract_version=2）："module:function(payload, source_ref)
cloudra/tools/registry.py:31        # -> list[EvidenceFact]"：从已登记证据中抽类型化事实（含反证）。
cloudra/tools/registry.py:32        dag_fact_adapter: str = ""
cloudra/tools/registry.py:33        # 证据产生工具：完成的调用产出计算证据（知识检索触发语义）。
cloudra/tools/registry.py:34        # §5 起不再有"自动入 CallReview 收件箱"的开关——送审只能显式
cloudra/tools/registry.py:35        # （dag_bind_evidence with claim）。
cloudra/tools/registry.py:36        dag_evidence_producer: bool = False
cloudra/tools/registry.py:37        # Phase 3 门槛：mechanism 未被接受前拦截的计算入口（开算/写计算输入）。
cloudra/tools/registry.py:38        dag_compute_entry: bool = False
cloudra/tools/registry.py:39        dag_compute_predicate: str = ""  # "module:function(arguments)->bool", overrides
dag_compute_entry
cloudra/tools/registry.py:40
cloudra/tools/registry.py:41        # Runtime availability. The default keeps the Android allow-list opt-in.
```

```
cloudra/tools/registry.py:42        runtime_profiles: frozenset[str] = frozenset({"desktop"})
cloudra/tools/registry.py:43
cloudra/tools/registry.py:44        # Visual-mode gate: image-producing/consuming tools hidden unless the
cloudra/tools/registry.py:45        # agent's visual_mode setting is on.
cloudra/tools/registry.py:46        visual: bool = False
cloudra/tools/registry.py:47
cloudra/tools/registry.py:48        # Supervisor capability: whether the scientific Supervisor may discover and
cloudra/tools/registry.py:49        # execute this tool. One of: "none" (default, hidden), "evidence_read"
cloudra/tools/registry.py:50        # (read-only evidence access), "skill_read", "hub" (AgentHub, action-gated),
cloudra/tools/registry.py:51        # "review_decision" (control-plane verdicts). Projected exclusively via
cloudra/tools/registry.py:52        # tools/capabilities.py; do not keep separate supervisor tool name sets.
cloudra/tools/registry.py:53        supervisor_access: str = "none"
cloudra/tools/registry.py:54
cloudra/tools/registry.py:55        # Invocation routing. Direct controls stay top-level wire tools in the
cloudra/tools/registry.py:56        # javascript protocol (exposed next to js_exec) and are rejected inside a
cloudra/tools/registry.py:57        # guest program; ordinary business tools are the only guest-callable ones.
cloudra/tools/registry.py:58        invocation_direct_only: bool = False
cloudra/tools/registry.py:59
cloudra/tools/registry.py:60        # Config gate: "<section>.<field>" naming a boolean on CloudraConfig.
cloudra/tools/registry.py:61        # When the flag is false the tool is hidden from the wire projection and
cloudra/tools/registry.py:62        # hard-fails at call time (the tool itself must check the flag too).
cloudra/tools/registry.py:63        config_gate: str = ""
cloudra/tools/registry.py:64
cloudra/tools/registry.py:65        # Role allowlist: non-empty restricts the tool to the listed role_ids —
cloudra/tools/registry.py:66        # the wire projection hides it from other roles and PermissionStage
cloudra/tools/registry.py:67        # hard-fails their calls. Empty means every role may use it (default).
cloudra/tools/registry.py:68        # Never set on direct controls, dag_* or supervisor-surface tools: their
cloudra/tools/registry.py:69        # gates (invocation_direct_only / orchestration mode / supervisor_access)
cloudra/tools/registry.py:70        # are separate mechanisms.
cloudra/tools/registry.py:71        role_allowlist: frozenset[str] = frozenset()
cloudra/tools/registry.py:72
cloudra/tools/registry.py:73
cloudra/tools/registry.py:74    # 化学/QM 工具的默认使用者：科学计算角色 + Coordinator（benchmark 题面
cloudra/tools/registry.py:75    # agent 以 coordinator 角色运行，绝不能失去化学工具链）。supervisor 在列
cloudra/tools/registry.py:76    # 仅为保住其 evidence_read 投影面（log_parser/reaction_mapping）——它的
cloudra/tools/registry.py:77    # 实际可见面仍由 supervisor_access 收窄，allowlist 对它是只减不增。
cloudra/tools/registry.py:78    _SCIENCE_ROLES = frozenset(
cloudra/tools/registry.py:79        {"coordinator", "small_mol", "md", "dft", "experiment", "supervisor"}
cloudra/tools/registry.py:80    )
cloudra/tools/registry.py:81
cloudra/tools/registry.py:82    TOOL_REGISTRY: tuple[ToolRegistration, ...] = (
cloudra/tools/registry.py:83        ToolRegistration(
cloudra/tools/registry.py:84            module='cloudra.tools.Bashtool.bash',
cloudra/tools/registry.py:85            class_name='Bash',
cloudra/tools/registry.py:86            tool_name='bash',
cloudra/tools/registry.py:87            dag_compute_predicate='cloudra.tools.core.bash_habit_guard:
is_substantive_bash_command',
cloudra/tools/registry.py:88        ),
cloudra/tools/registry.py:89        ToolRegistration(
cloudra/tools/registry.py:90            module='cloudra.tools.Browsertool.browser',
cloudra/tools/registry.py:91            class_name='HeadlessBrowser',
cloudra/tools/registry.py:92            tool_name='browser',
cloudra/tools/registry.py:93        ),
```

```
cloudra/tools/registry.py:94        ToolRegistration(
cloudra/tools/registry.py:95            module='cloudra.tools.Chemtool.conformer_search',
cloudra/tools/registry.py:96            class_name='ConformerSearch',
cloudra/tools/registry.py:97            tool_name='conformer_search',
cloudra/tools/registry.py:98            dag_compute_entry=True,
cloudra/tools/registry.py:99            role_allowlist=_SCIENCE_ROLES,
cloudra/tools/registry.py:100       ),
cloudra/tools/registry.py:101       ToolRegistration(
cloudra/tools/registry.py:102           module='cloudra.tools.Chemtool.electronic_analysis',
cloudra/tools/registry.py:103           class_name='ElectronicAnalysis',
cloudra/tools/registry.py:104           tool_name='electronic_analysis',
cloudra/tools/registry.py:105           dag_compute_entry=True,
cloudra/tools/registry.py:106           role_allowlist=_SCIENCE_ROLES,
cloudra/tools/registry.py:107       ),
cloudra/tools/registry.py:108       ToolRegistration(
cloudra/tools/registry.py:109           module='cloudra.tools.Chemtool.format_converter',
cloudra/tools/registry.py:110           class_name='FormatConverter',
cloudra/tools/registry.py:111           tool_name='format_converter',
cloudra/tools/registry.py:112           role_allowlist=_SCIENCE_ROLES,
cloudra/tools/registry.py:113       ),
cloudra/tools/registry.py:114       ToolRegistration(
cloudra/tools/registry.py:115           module='cloudra.tools.Chemtool.qm_input_writer',
cloudra/tools/registry.py:116           class_name='QmInputWriter',
cloudra/tools/registry.py:117           tool_name='qm_input_writer',
cloudra/tools/registry.py:118           dag_compute_entry=True,
cloudra/tools/registry.py:119           role_allowlist=_SCIENCE_ROLES,
cloudra/tools/registry.py:120       ),
cloudra/tools/registry.py:121       ToolRegistration(
cloudra/tools/registry.py:122           module='cloudra.tools.Chemtool.image_to_smiles',
cloudra/tools/registry.py:123           class_name='ImageToSmiles',
cloudra/tools/registry.py:124           tool_name='image_to_smiles',
cloudra/tools/registry.py:125           role_allowlist=_SCIENCE_ROLES,
cloudra/tools/registry.py:126       ),
cloudra/tools/registry.py:127       ToolRegistration(
cloudra/tools/registry.py:128           module='cloudra.tools.Chemtool.cdxml_to_graph',
cloudra/tools/registry.py:129           class_name='CdxmlToGraph',
cloudra/tools/registry.py:130           tool_name='cdxml_to_graph',
cloudra/tools/registry.py:131           role_allowlist=_SCIENCE_ROLES,
cloudra/tools/registry.py:132       ),
cloudra/tools/registry.py:133       ToolRegistration(
cloudra/tools/registry.py:134           module='cloudra.tools.Chemtool.internal_coordinates',
cloudra/tools/registry.py:135           class_name='InternalCoordinates',
cloudra/tools/registry.py:136           tool_name='internal_coordinates',
cloudra/tools/registry.py:137           role_allowlist=_SCIENCE_ROLES,
cloudra/tools/registry.py:138       ),
cloudra/tools/registry.py:139       ToolRegistration(
cloudra/tools/registry.py:140           module='cloudra.tools.Chemtool.ir_processor',
cloudra/tools/registry.py:141           class_name='IRProcessor',
cloudra/tools/registry.py:142           tool_name='ir_processor',
cloudra/tools/registry.py:143           # 实验谱图处理：experiment 角色专属
cloudra/tools/registry.py:144           role_allowlist=frozenset({'experiment'}),
cloudra/tools/registry.py:145       ),
```

```
cloudra/tools/registry.py:146       ToolRegistration(
cloudra/tools/registry.py:147           module='cloudra.tools.Chemtool.log_parser',
cloudra/tools/registry.py:148           class_name='LogParser',
cloudra/tools/registry.py:149           tool_name='log_parser',
cloudra/tools/registry.py:150           evidence_deriver='cloudra.tools.Chemtool.log_parser:derive_delivery_changes',
cloudra/tools/registry.py:151           dag_fact_adapter='cloudra.dag.fact_adapters:log_parser_facts',
cloudra/tools/registry.py:152           dag_evidence_producer=True,
cloudra/tools/registry.py:153           supervisor_access='evidence_read',
cloudra/tools/registry.py:154           role_allowlist=_SCIENCE_ROLES,
cloudra/tools/registry.py:155       ),
cloudra/tools/registry.py:156       ToolRegistration(
cloudra/tools/registry.py:157           module='cloudra.tools.Chemtool.molecule_drawer',
cloudra/tools/registry.py:158           class_name='MoleculeDrawer',
cloudra/tools/registry.py:159           tool_name='molecule_drawer',
cloudra/tools/registry.py:160           role_allowlist=_SCIENCE_ROLES,
cloudra/tools/registry.py:161       ),
cloudra/tools/registry.py:162       ToolRegistration(
cloudra/tools/registry.py:163           module='cloudra.tools.Chemtool.mechanism_predict',
cloudra/tools/registry.py:164           class_name='MechanismPredict',
cloudra/tools/registry.py:165           tool_name='mechanism_predict',
cloudra/tools/registry.py:166           # ML 机理假设生成器（FlowER 外部 conda env）：不是计算证据，默认
cloudra/tools/registry.py:167           # 由 flower.enabled 开关关闭。
cloudra/tools/registry.py:168           config_gate='flower.enabled',
cloudra/tools/registry.py:169           role_allowlist=_SCIENCE_ROLES,
cloudra/tools/registry.py:170       ),
cloudra/tools/registry.py:171       ToolRegistration(
cloudra/tools/registry.py:172           module='cloudra.tools.Chemtool.name_to_structure',
cloudra/tools/registry.py:173           class_name='NameToStructure',
cloudra/tools/registry.py:174           tool_name='name_to_structure',
cloudra/tools/registry.py:175           role_allowlist=_SCIENCE_ROLES,
cloudra/tools/registry.py:176       ),
cloudra/tools/registry.py:177       ToolRegistration(
cloudra/tools/registry.py:178           module='cloudra.tools.Chemtool.nmr_processor',
cloudra/tools/registry.py:179           class_name='NMRProcessor',
cloudra/tools/registry.py:180           tool_name='nmr_processor',
cloudra/tools/registry.py:181           # 实验谱图处理：experiment 角色专属
cloudra/tools/registry.py:182           role_allowlist=frozenset({'experiment'}),
cloudra/tools/registry.py:183       ),
cloudra/tools/registry.py:184       ToolRegistration(
cloudra/tools/registry.py:185           module='cloudra.tools.Chemtool.pymol_render',
cloudra/tools/registry.py:186           class_name='PymolRender',
cloudra/tools/registry.py:187           tool_name='pymol_render',
cloudra/tools/registry.py:188           visual=True,
cloudra/tools/registry.py:189           role_allowlist=_SCIENCE_ROLES,
cloudra/tools/registry.py:190       ),
cloudra/tools/registry.py:191       ToolRegistration(
cloudra/tools/registry.py:192           module='cloudra.tools.Chemtool.query_compute_job',
cloudra/tools/registry.py:193           class_name='QueryComputeJob',
cloudra/tools/registry.py:194           tool_name='query_compute_job',
cloudra/tools/registry.py:195           role_allowlist=_SCIENCE_ROLES,
cloudra/tools/registry.py:196       ),
cloudra/tools/registry.py:197       ToolRegistration(
```

```
cloudra/tools/registry.py:198           module='cloudra.tools.Chemtool.query_compute_queue',
cloudra/tools/registry.py:199           class_name='QueryComputeQueue',
cloudra/tools/registry.py:200           tool_name='query_compute_queue',
cloudra/tools/registry.py:201           role_allowlist=_SCIENCE_ROLES,
cloudra/tools/registry.py:202       ),
cloudra/tools/registry.py:203       ToolRegistration(
cloudra/tools/registry.py:204           module='cloudra.tools.Chemtool.mark_compute_job_failed',
cloudra/tools/registry.py:205           class_name='MarkComputeJobFailed',
cloudra/tools/registry.py:206           tool_name='mark_compute_job_failed',
cloudra/tools/registry.py:207           dag_observe=False,
cloudra/tools/registry.py:208           role_allowlist=_SCIENCE_ROLES,
cloudra/tools/registry.py:209       ),
cloudra/tools/registry.py:210       ToolRegistration(
cloudra/tools/registry.py:211           module='cloudra.tools.Chemtool.reaction_mapping',
cloudra/tools/registry.py:212           class_name='ReactionMapping',
cloudra/tools/registry.py:213           tool_name='reaction_mapping',
cloudra/tools/registry.py:214           evidence_deriver='cloudra.tools.Chemtool.reaction_mapping:derive_delivery_changes',
cloudra/tools/registry.py:215           dag_fact_adapter='cloudra.dag.fact_adapters:reaction_mapping_facts',
cloudra/tools/registry.py:216           dag_evidence_producer=True,
cloudra/tools/registry.py:217           supervisor_access='evidence_read',
cloudra/tools/registry.py:218           role_allowlist=_SCIENCE_ROLES,
cloudra/tools/registry.py:219       ),
cloudra/tools/registry.py:220       # TS 搜索 SOP 的 agent 可见接口（只有这两个函数：刻意没有 status/查询接口）
cloudra/tools/registry.py:221       ToolRegistration(
cloudra/tools/registry.py:222           'cloudra.tools.Chemtool.ts_search_process', 'TsPrepare', tool_name='ts_prepare',
cloudra/tools/registry.py:223           role_allowlist=_SCIENCE_ROLES,
cloudra/tools/registry.py:224       ),
cloudra/tools/registry.py:225       ToolRegistration(
cloudra/tools/registry.py:226           'cloudra.tools.Chemtool.ts_search_process', 'TsResume', tool_name='ts_resume',
cloudra/tools/registry.py:227           role_allowlist=_SCIENCE_ROLES,
cloudra/tools/registry.py:228       ),
cloudra/tools/registry.py:229       ToolRegistration(
cloudra/tools/registry.py:230           module='cloudra.tools.Chemtool.smiles_to_mol',
cloudra/tools/registry.py:231           class_name='SmilesToMol',
cloudra/tools/registry.py:232           tool_name='smiles_to_mol',
cloudra/tools/registry.py:233           role_allowlist=_SCIENCE_ROLES,
cloudra/tools/registry.py:234       ),
cloudra/tools/registry.py:235       ToolRegistration(
cloudra/tools/registry.py:236           module='cloudra.tools.Chemtool.submit_compute_job',
cloudra/tools/registry.py:237           class_name='SubmitComputeJob',
cloudra/tools/registry.py:238           tool_name='submit_compute_job',
cloudra/tools/registry.py:239           dag_compute_entry=True,
cloudra/tools/registry.py:240           role_allowlist=_SCIENCE_ROLES,
cloudra/tools/registry.py:241       ),
cloudra/tools/registry.py:242       ToolRegistration(
cloudra/tools/registry.py:243           module='cloudra.tools.Chemtool.thermo_analysis',
cloudra/tools/registry.py:244           class_name='ThermoAnalysis',
cloudra/tools/registry.py:245           tool_name='thermo_analysis',
cloudra/tools/registry.py:246           evidence_deriver='cloudra.tools.Chemtool.thermo_analysis:derive_delivery_changes',
cloudra/tools/registry.py:247           dag_evidence_producer=True,
cloudra/tools/registry.py:248           role_allowlist=_SCIENCE_ROLES,
cloudra/tools/registry.py:249       ),
```

```
        ToolRegistration(
            module='cloudra.tools.Chemtool.ts_guess',
            class_name='TsGuess',
            tool_name='ts_guess',
            evidence_deriver='cloudra.tools.Chemtool.ts_guess:derive_delivery_changes',
            dag_fact_adapter='cloudra.dag.fact_adapters:ts_guess_facts',
            dag_evidence_producer=True,
            dag_compute_predicate='cloudra.tools.Chemtool.ts_guess:is_compute_entry_call',
            role_allowlist=_SCIENCE_ROLES,
        ),
        ToolRegistration(
            module='cloudra.tools.Interactool.ask_user',
            class_name='AskUser',
            tool_name='ask_user',
            invocation_direct_only=True,
        ),
        ToolRegistration(
            module='cloudra.tools.Interactool.email_reader',
            class_name='EmailReader',
            tool_name='email_reader',
            config_gate='email.enabled',
        ),
        ToolRegistration(
            module='cloudra.tools.Interactool.email_sender',
            class_name='EmailSender',
            tool_name='email_sender',
            config_gate='email.enabled',
        ),
        ToolRegistration(
            module='cloudra.tools.Managetool.agent_hub',
            class_name='AgentHub',
            tool_name='agent_hub',
            runtime_profiles=frozenset({'desktop', 'android'}),
            supervisor_access='hub',
        ),
        ToolRegistration(
            module='cloudra.tools.Managetool.get_subagent_report',
            class_name='GetSubAgentReport',
            tool_name='get_subagent_report',
            runtime_profiles=frozenset({'desktop', 'android'}),
        ),
        ToolRegistration(
            module='cloudra.tools.Managetool.schedule',
            class_name='Schedule',
            tool_name='schedule',
            runtime_profiles=frozenset({'desktop', 'android'}),
            invocation_direct_only=True,
        ),
        ToolRegistration(
            module='cloudra.tools.Managetool.subagent_launcher',
            class_name='SubAgentLauncher',
            tool_name='subagent_launcher',
```

```
302 cloudra/tools/registry.py:302           runtime_profiles=frozenset({'desktop', 'android'}),
303 cloudra/tools/registry.py:303       ),
304 cloudra/tools/registry.py:304       ToolRegistration(
305 cloudra/tools/registry.py:305           module='cloudra.tools.Managetool.subagent_swarm',
306 cloudra/tools/registry.py:306           class_name='SubAgentSwarm',
307 cloudra/tools/registry.py:307           tool_name='subagent_swarm',
308 cloudra/tools/registry.py:308       ),
309 cloudra/tools/registry.py:309       ToolRegistration('cloudra.tools.Plantool.goal', 'GoalSet'),
310 cloudra/tools/registry.py:310       ToolRegistration(
311 cloudra/tools/registry.py:311           module='cloudra.tools.Plantool.dag_tools.dag_view',
312 cloudra/tools/registry.py:312           class_name='DagView',
313 cloudra/tools/registry.py:313           tool_name='dag_view',
314 cloudra/tools/registry.py:314           dag_observe=False,
315 cloudra/tools/registry.py:315           supervisor_access='evidence_read',
316 cloudra/tools/registry.py:316       ),
317 cloudra/tools/registry.py:317       ToolRegistration(
318 cloudra/tools/registry.py:318           module='cloudra.tools.Plantool.dag_tools.dag_delivery_report',
319 cloudra/tools/registry.py:319           class_name='DagDeliveryReport',
320 cloudra/tools/registry.py:320           tool_name='dag_delivery_report',
321 cloudra/tools/registry.py:321           dag_observe=False,
322 cloudra/tools/registry.py:322           supervisor_access='evidence_read',
323 cloudra/tools/registry.py:323       ),
324 cloudra/tools/registry.py:324       ToolRegistration(
325 cloudra/tools/registry.py:325           module='cloudra.tools.Plantool.dag_tools.dag_add_species',
326 cloudra/tools/registry.py:326           class_name='DagAddSpecies',
327 cloudra/tools/registry.py:327           tool_name='dag_add_species',
328 cloudra/tools/registry.py:328           dag_observe=False,
329 cloudra/tools/registry.py:329       ),
330 cloudra/tools/registry.py:330       ToolRegistration(
331 cloudra/tools/registry.py:331           module='cloudra.tools.Plantool.dag_tools.dag_add_step',
332 cloudra/tools/registry.py:332           class_name='DagAddStep',
333 cloudra/tools/registry.py:333           tool_name='dag_add_step',
334 cloudra/tools/registry.py:334           dag_observe=False,
335 cloudra/tools/registry.py:335       ),
336 cloudra/tools/registry.py:336       ToolRegistration(
337 cloudra/tools/registry.py:337           module='cloudra.tools.Plantool.dag_tools.dag_update_delivery_element',
338 cloudra/tools/registry.py:338           class_name='DagUpdateDeliveryElement',
339 cloudra/tools/registry.py:339           tool_name='dag_update_delivery_element',
340 cloudra/tools/registry.py:340           dag_observe=False,
341 cloudra/tools/registry.py:341       ),
342 cloudra/tools/registry.py:342       ToolRegistration(
343 cloudra/tools/registry.py:343           module='cloudra.tools.Plantool.dag_tools.dag_remove_edge',
344 cloudra/tools/registry.py:344           class_name='DagRemoveEdge',
345 cloudra/tools/registry.py:345           tool_name='dag_remove_edge',
346 cloudra/tools/registry.py:346           dag_observe=False,
347 cloudra/tools/registry.py:347       ),
348 cloudra/tools/registry.py:348       ToolRegistration(
349 cloudra/tools/registry.py:349           module='cloudra.tools.Plantool.dag_tools.dag_remove_node',
350 cloudra/tools/registry.py:350           class_name='DagRemoveNode',
351 cloudra/tools/registry.py:351           tool_name='dag_remove_node',
352 cloudra/tools/registry.py:352           dag_observe=False,
353 cloudra/tools/registry.py:353           # 无 supervisor_access：删图是研究侧（Coordinator/子 agent）的修改
```

```
cloudra/tools/registry.py:354            # 权限：Supervisor 保留审查职责、不顺带获得直接删图能力。
cloudra/tools/registry.py:355        ),
cloudra/tools/registry.py:356        ToolRegistration(
cloudra/tools/registry.py:357            module='cloudra.tools.Plantool.dag_tools.dag_bind_evidence',
cloudra/tools/registry.py:358            class_name='DagBindEvidence',
cloudra/tools/registry.py:359            tool_name='dag_bind_evidence',
cloudra/tools/registry.py:360            dag_observe=False,
cloudra/tools/registry.py:361        ),
cloudra/tools/registry.py:362        ToolRegistration(
cloudra/tools/registry.py:363            module='cloudra.tools.Plantool.dag_tools.dag_propose_mechanism',
cloudra/tools/registry.py:364            class_name='DagProposeMechanism',
cloudra/tools/registry.py:365            tool_name='dag_propose_mechanism',
cloudra/tools/registry.py:366            dag_observe=False,
cloudra/tools/registry.py:367        ),
cloudra/tools/registry.py:368        ToolRegistration(
cloudra/tools/registry.py:369            module='cloudra.tools.Plantool.dag_tools.dag_submit_step_evidence',
cloudra/tools/registry.py:370            class_name='DagSubmitStepEvidence',
cloudra/tools/registry.py:371            tool_name='dag_submit_step_evidence',
cloudra/tools/registry.py:372            dag_observe=False,
cloudra/tools/registry.py:373        ),
cloudra/tools/registry.py:374        ToolRegistration(
cloudra/tools/registry.py:375            module='cloudra.tools.Plantool.dag_tools.dag_submit_species_evidence',
cloudra/tools/registry.py:376            class_name='DagSubmitSpeciesEvidence',
cloudra/tools/registry.py:377            tool_name='dag_submit_species_evidence',
cloudra/tools/registry.py:378            dag_observe=False,
cloudra/tools/registry.py:379        ),
cloudra/tools/registry.py:380        ToolRegistration(
cloudra/tools/registry.py:381            module='cloudra.tools.Plantool.dag_tools.dag_review_step',
cloudra/tools/registry.py:382            class_name='DagReviewStep',
cloudra/tools/registry.py:383            tool_name='dag_review_step',
cloudra/tools/registry.py:384            dag_observe=False,
cloudra/tools/registry.py:385            supervisor_access='review_decision',
cloudra/tools/registry.py:386        ),
cloudra/tools/registry.py:387        ToolRegistration(
cloudra/tools/registry.py:388            module='cloudra.tools.Plantool.dag_tools.dag_review_species',
cloudra/tools/registry.py:389            class_name='DagReviewSpecies',
cloudra/tools/registry.py:390            tool_name='dag_review_species',
cloudra/tools/registry.py:391            dag_observe=False,
cloudra/tools/registry.py:392            supervisor_access='review_decision',
cloudra/tools/registry.py:393        ),
cloudra/tools/registry.py:394        ToolRegistration(
cloudra/tools/registry.py:395            module='cloudra.tools.Plantool.dag_tools.dag_accept_mechanism',
cloudra/tools/registry.py:396            class_name='DagAcceptMechanism',
cloudra/tools/registry.py:397            tool_name='dag_accept_mechanism',
cloudra/tools/registry.py:398            dag_observe=False,
cloudra/tools/registry.py:399            supervisor_access='review_decision',
cloudra/tools/registry.py:400        ),
cloudra/tools/registry.py:401        ToolRegistration(
cloudra/tools/registry.py:402            module='cloudra.tools.Plantool.dag_tools.dag_review_call',
cloudra/tools/registry.py:403            class_name='DagReviewCall',
cloudra/tools/registry.py:404            tool_name='dag_review_call',
cloudra/tools/registry.py:405            dag_observe=False,
```

```
cloudra/tools/registry.py:406           supervisor_access='review_decision',
cloudra/tools/registry.py:407       ),
cloudra/tools/registry.py:408       ToolRegistration(
cloudra/tools/registry.py:409           module='cloudra.tools.Plantool.dag_tools.dag_review_evidence',
cloudra/tools/registry.py:410           class_name='DagReviewEvidence',
cloudra/tools/registry.py:411           tool_name='dag_review_evidence',
cloudra/tools/registry.py:412           dag_observe=False,
cloudra/tools/registry.py:413           supervisor_access='review_decision',
cloudra/tools/registry.py:414       ),
cloudra/tools/registry.py:415       ToolRegistration(
cloudra/tools/registry.py:416           module='cloudra.tools.Plantool.dag_tools.dag_view_evidence',
cloudra/tools/registry.py:417           class_name='DagViewEvidence',
cloudra/tools/registry.py:418           tool_name='dag_view_evidence',
cloudra/tools/registry.py:419           dag_observe=False,
cloudra/tools/registry.py:420           supervisor_access='evidence_read',
cloudra/tools/registry.py:421       ),
cloudra/tools/registry.py:422       ToolRegistration(
cloudra/tools/registry.py:423           module='cloudra.tools.Plantool.dag_tools.dag_set_coverage_contract',
cloudra/tools/registry.py:424           class_name='DagSetCoverageContract',
cloudra/tools/registry.py:425           tool_name='dag_set_coverage_contract',
cloudra/tools/registry.py:426           dag_observe=False,
cloudra/tools/registry.py:427           supervisor_access='review_decision',
cloudra/tools/registry.py:428       ),
cloudra/tools/registry.py:429       ToolRegistration(
cloudra/tools/registry.py:430           module='cloudra.tools.Plantool.dag_tools.dag_message_supervisor',
cloudra/tools/registry.py:431           class_name='DagMessageSupervisor',
cloudra/tools/registry.py:432           tool_name='dag_message_supervisor',
cloudra/tools/registry.py:433           dag_observe=False,
cloudra/tools/registry.py:434           # 无 supervisor_access：这是研究 Agent/Coordinator → Supervisor 的
cloudra/tools/registry.py:435           # 请求入口，工具本身硬拒 Supervisor 调用；督导的回复路径是
cloudra/tools/registry.py:436           # dag_review_* 裁决工具与运行结束报告。
cloudra/tools/registry.py:437       ),
cloudra/tools/registry.py:438       ToolRegistration('cloudra.tools.Plantool.todo', 'TodoWrite'),
cloudra/tools/registry.py:439       ToolRegistration(
cloudra/tools/registry.py:440           module='cloudra.tools.Searchtool.read',
cloudra/tools/registry.py:441           class_name='Read',
cloudra/tools/registry.py:442           tool_name='read',
cloudra/tools/registry.py:443           runtime_profiles=frozenset({'desktop', 'android'}),
cloudra/tools/registry.py:444           supervisor_access='evidence_read',
cloudra/tools/registry.py:445       ),
cloudra/tools/registry.py:446       ToolRegistration(
cloudra/tools/registry.py:447           module='cloudra.tools.Searchtool.search',
cloudra/tools/registry.py:448           class_name='Search',
cloudra/tools/registry.py:449           tool_name='search',
cloudra/tools/registry.py:450           runtime_profiles=frozenset({'desktop', 'android'}),
cloudra/tools/registry.py:451       ),
cloudra/tools/registry.py:452       ToolRegistration(
cloudra/tools/registry.py:453           module='cloudra.tools.Skilltool.get_skill',
cloudra/tools/registry.py:454           class_name='GetSkill',
cloudra/tools/registry.py:455           tool_name='get_skill',
cloudra/tools/registry.py:456           runtime_profiles=frozenset({'desktop', 'android'}),
cloudra/tools/registry.py:457           dag_observe=False,
```

```
cloudra/tools/registry.py:458           supervisor_access='skill_read',
cloudra/tools/registry.py:459       ),
cloudra/tools/registry.py:460       ToolRegistration(
cloudra/tools/registry.py:461           module='cloudra.tools.Stoptool.stop_tool',
cloudra/tools/registry.py:462           class_name='StopTool',
cloudra/tools/registry.py:463           tool_name='stop',
cloudra/tools/registry.py:464           runtime_profiles=frozenset({'desktop', 'android'}),
cloudra/tools/registry.py:465           invocation_direct_only=True,
cloudra/tools/registry.py:466       ),
cloudra/tools/registry.py:467       ToolRegistration(
cloudra/tools/registry.py:468           module='cloudra.tools.Writetool.edit',
cloudra/tools/registry.py:469           class_name='Edit',
cloudra/tools/registry.py:470           tool_name='edit',
cloudra/tools/registry.py:471           runtime_profiles=frozenset({'desktop', 'android'}),
cloudra/tools/registry.py:472       ),
cloudra/tools/registry.py:473       ToolRegistration(
cloudra/tools/registry.py:474           module='cloudra.tools.Writetool.write',
cloudra/tools/registry.py:475           class_name='Write',
cloudra/tools/registry.py:476           tool_name='write',
cloudra/tools/registry.py:477           runtime_profiles=frozenset({'desktop', 'android'}),
cloudra/tools/registry.py:478       ),
cloudra/tools/registry.py:479       ToolRegistration(
cloudra/tools/registry.py:480           module='cloudra.tools.Writetool.update_memory',
cloudra/tools/registry.py:481           class_name='UpdateMemory',
cloudra/tools/registry.py:482           tool_name='update_memory',
cloudra/tools/registry.py:483           runtime_profiles=frozenset({'desktop', 'android'}),
cloudra/tools/registry.py:484       ),
cloudra/tools/registry.py:485   )
cloudra/web_launcher.py:1   """Web frontend launcher used by the `cloudra` command.
cloudra/web_launcher.py:2
cloudra/web_launcher.py:3   Keeps Node/npm knowledge out of the CLI module.  The launcher works for source
cloudra/web_launcher.py:4   checkouts and editable installs; packaged wheels can point CLOUDRA_FRONTEND_DIR
cloudra/web_launcher.py:5   at an unpacked frontend directory.
cloudra/web_launcher.py:6   """
cloudra/web_launcher.py:7
cloudra/web_launcher.py:8   from __future__ import annotations
cloudra/web_launcher.py:9
cloudra/web_launcher.py:10  import logging
cloudra/web_launcher.py:11  import os
cloudra/web_launcher.py:12  import shutil
cloudra/web_launcher.py:13  import socket
cloudra/web_launcher.py:14  import subprocess
cloudra/web_launcher.py:15  import sys
cloudra/web_launcher.py:16  import time
cloudra/web_launcher.py:17  import urllib.request
cloudra/web_launcher.py:18  import webbrowser
cloudra/web_launcher.py:19  from pathlib import Path
cloudra/web_launcher.py:20
cloudra/web_launcher.py:21  from cloudra.services.daemon_supervisor import ensure_daemons
cloudra/web_launcher.py:22
cloudra/web_launcher.py:23  # 常驻 SSE 不会自己结束（SPA 一打开就挂着 EventSource，见 frontend/src/api.ts），
cloudra/web_launcher.py:24  # 而 uvicorn 默认（None）会**无限等待活跃连接**：`cloudra stop` 因此报「API: 已停止」
```

```
cloudra/web_launcher.py:25  # 却关不死进程（GH #38）。给一个有界上界，超时后由 uvicorn 取消这些连接——流生成器
cloudra/web_launcher.py:26  # 的 finally 照常退订。取值与本文件的 _terminate_runtime(grace=20) 对齐。
cloudra/web_launcher.py:27  _GRACEFUL_SHUTDOWN_TIMEOUT_SECONDS = 20
cloudra/web_launcher.py:28
cloudra/web_launcher.py:29  # 默认只绑回环。`--host` 可显式改绑（局域网/SSH 远程访问场景）；绑定地址经
cloudra/web_launcher.py:30  # CLOUDRA_API_BIND_HOST 传进 uvicorn 子进程，api_v2 的 Host 白名单据此放开
cloudra/web_launcher.py:31  # （见 app.py::_host_allowlist）。
cloudra/web_launcher.py:32  _DEFAULT_BIND_HOST = "127.0.0.1"
cloudra/web_launcher.py:33  _LOOPBACK_BIND_HOSTS = {"127.0.0.1", "localhost", "::1", "[::1]"}
cloudra/web_launcher.py:34  _WILDCARD_BIND_HOSTS = {"0.0.0.0", "::", "*"}
cloudra/web_launcher.py:35
cloudra/web_launcher.py:36  logger = logging.getLogger(__name__)
cloudra/web_launcher.py:37
cloudra/web_launcher.py:38
cloudra/web_launcher.py:39  def _frontend_dir() -> Path:
cloudra/web_launcher.py:40      override = os.environ.get("CLOUDRA_FRONTEND_DIR", "").strip()
cloudra/web_launcher.py:41      candidates = []
cloudra/web_launcher.py:42      if override:
cloudra/web_launcher.py:43          candidates.append(Path(override))
cloudra/web_launcher.py:44      # cloudra/web_launcher.py -> cloudra/.. == repository root (source checkout)
cloudra/web_launcher.py:45      candidates.append(Path(__file__).resolve().parents[1] / "frontend")
cloudra/web_launcher.py:46      candidates.append(Path.cwd() / "frontend")
cloudra/web_launcher.py:47      for candidate in candidates:
cloudra/web_launcher.py:48          if (candidate / "package.json").is_file():
cloudra/web_launcher.py:49              return candidate.resolve()
cloudra/web_launcher.py:50      raise RuntimeError(
cloudra/web_launcher.py:51          "frontend directory not found; set CLOUDRA_FRONTEND_DIR to the "
cloudra/web_launcher.py:52          "directory containing frontend/package.json"
cloudra/web_launcher.py:53      )
cloudra/web_launcher.py:54
cloudra/web_launcher.py:55
cloudra/web_launcher.py:56  def _check_node(frontend: Path) -> None:
cloudra/web_launcher.py:57      npm = shutil.which("npm")
cloudra/web_launcher.py:58      node = shutil.which("node")
cloudra/web_launcher.py:59      if npm is None or node is None:
cloudra/web_launcher.py:60          raise RuntimeError(
cloudra/web_launcher.py:61              "Node.js >= 18 and npm are required for the Cloudra web frontend. "
cloudra/web_launcher.py:62              f"Frontend found at {frontend}. Install from https://nodejs.org/"
cloudra/web_launcher.py:63          )
cloudra/web_launcher.py:64
cloudra/web_launcher.py:65
cloudra/web_launcher.py:66  def _npm_argv() -> list[str]:
cloudra/web_launcher.py:67      """npm as an invokable argv prefix.
cloudra/web_launcher.py:68
cloudra/web_launcher.py:69      On Windows npm is a ``npm.cmd`` batch file: ``shutil.which`` finds it (it
cloudra/web_launcher.py:70      honours PATHEXT) but ``CreateProcess`` cannot execute ``.cmd`` scripts
cloudra/web_launcher.py:71      directly — ``subprocess.run(["npm", ...])`` dies with ``WinError 2`` even
cloudra/web_launcher.py:72      though the pre-check passed.  Route through ``cmd /c`` with the fully
cloudra/web_launcher.py:73      resolved path; POSIX keeps the resolved executable directly.
cloudra/web_launcher.py:74      """
cloudra/web_launcher.py:75      npm = shutil.which("npm")
cloudra/web_launcher.py:76      if npm is None:
```

```
cloudra/web_launcher.py:77          raise RuntimeError(
cloudra/web_launcher.py:78              "npm not found on PATH; install Node.js >= 18 or run with --skip-build"
cloudra/web_launcher.py:79          )
cloudra/web_launcher.py:80      if os.name == "nt":
cloudra/web_launcher.py:81          return ["cmd", "/c", npm]
cloudra/web_launcher.py:82      return [npm]
cloudra/web_launcher.py:83
cloudra/web_launcher.py:84
cloudra/web_launcher.py:85  def _frontend_source_newer_than_dist(frontend: Path) -> bool:
cloudra/web_launcher.py:86      dist_index = frontend / "dist" / "index.html"
cloudra/web_launcher.py:87      if not dist_index.is_file():
cloudra/web_launcher.py:88          return True
cloudra/web_launcher.py:89      dist_mtime = dist_index.stat().st_mtime
cloudra/web_launcher.py:90      for root in (frontend / "src", frontend / "public"):
cloudra/web_launcher.py:91          if not root.is_dir():
cloudra/web_launcher.py:92              continue
cloudra/web_launcher.py:93          for path in root.rglob("*"):
cloudra/web_launcher.py:94              if path.is_file() and path.stat().st_mtime > dist_mtime:
cloudra/web_launcher.py:95                  return True
cloudra/web_launcher.py:96      for name in ("index.html", "vite.config.ts", "package.json"):
cloudra/web_launcher.py:97          path = frontend / name
cloudra/web_launcher.py:98          if path.is_file() and path.stat().st_mtime > dist_mtime:
cloudra/web_launcher.py:99              return True
cloudra/web_launcher.py:100     return False
cloudra/web_launcher.py:101
cloudra/web_launcher.py:102
cloudra/web_launcher.py:103 def _ensure_frontend(frontend: Path, *, rebuild: bool = False, skip_build: bool = False) ->
    None:
cloudra/web_launcher.py:104     _check_node(frontend)
cloudra/web_launcher.py:105     if not (frontend / "node_modules").is_dir():
cloudra/web_launcher.py:106         print("[cloudra] installing frontend dependencies...", flush=True)
cloudra/web_launcher.py:107         subprocess.run(_npm_argv() + ["install", "--no-audit", "--no-fund"], cwd=frontend,
    check=True)
cloudra/web_launcher.py:108     dist = frontend / "dist"
cloudra/web_launcher.py:109     if skip_build:
cloudra/web_launcher.py:110         if not (dist / "index.html").is_file():
cloudra/web_launcher.py:111             raise RuntimeError("--skip-build was requested but frontend/dist is missing")
cloudra/web_launcher.py:112         return
cloudra/web_launcher.py:113     stale = _frontend_source_newer_than_dist(frontend)
cloudra/web_launcher.py:114     if rebuild or not (dist / "index.html").is_file() or stale:
cloudra/web_launcher.py:115         if stale and not rebuild and (dist / "index.html").is_file():
cloudra/web_launcher.py:116             print("[cloudra] frontend source changed — rebuilding...", flush=True)
cloudra/web_launcher.py:117         else:
cloudra/web_launcher.py:118             print("[cloudra] building frontend...", flush=True)
cloudra/web_launcher.py:119         subprocess.run(_npm_argv() + ["run", "build"], cwd=frontend, check=True)
cloudra/web_launcher.py:120
cloudra/web_launcher.py:121
cloudra/web_launcher.py:122 def _probe_host(bind_host: str) -> str:
cloudra/web_launcher.py:123     """本机侧探测/访问用的地址。
cloudra/web_launcher.py:124
cloudra/web_launcher.py:125     通配绑定（0.0.0.0/::）在回环上同样可达，用回环探测；绑具体网卡地址时
cloudra/web_launcher.py:126     回环上根本没有监听，必须用该地址本身。IPv6 字面量加方括号才能拼进 URL。
cloudra/web_launcher.py:127     """
cloudra/web_launcher.py:128     if bind_host in _WILDCARD_BIND_HOSTS:
```

```
cloudra/web_launcher.py:129         return "127.0.0.1"
cloudra/web_launcher.py:130     if ":" in bind_host:
cloudra/web_launcher.py:131         return f"[{bind_host}]"
cloudra/web_launcher.py:132     return bind_host
cloudra/web_launcher.py:133
cloudra/web_launcher.py:134
cloudra/web_launcher.py:135 def _is_loopback_bind(bind_host: str) -> bool:
cloudra/web_launcher.py:136     return bind_host in _LOOPBACK_BIND_HOSTS
cloudra/web_launcher.py:137
cloudra/web_launcher.py:138
cloudra/web_launcher.py:139 def _may_reuse_runtime(bind_host: str) -> bool:
cloudra/web_launcher.py:140     """能否复用已在跑的实例。
cloudra/web_launcher.py:141
cloudra/web_launcher.py:142     通配绑定不能：探测走回环，而回环探测分辨不出对端绑的是 127.0.0.1 还是
cloudra/web_launcher.py:143     0.0.0.0——复用会让 `--host 0.0.0.0` 静默失效（用户以为已对外可访问，
cloudra/web_launcher.py:144     实际只绑在回环上）。回环/具体地址的探测地址就是绑定地址本身，判定可信。
cloudra/web_launcher.py:145     """
cloudra/web_launcher.py:146     return bind_host not in _WILDCARD_BIND_HOSTS
cloudra/web_launcher.py:147
cloudra/web_launcher.py:148
cloudra/web_launcher.py:149 def _port_is_bindable(bind_host: str, preferred: int) -> bool:
cloudra/web_launcher.py:150     family = socket.AF_INET6 if ":" in bind_host else socket.AF_INET
cloudra/web_launcher.py:151     with socket.socket(family, socket.SOCK_STREAM) as sock:
cloudra/web_launcher.py:152         try:
cloudra/web_launcher.py:153             sock.bind((bind_host, preferred))
cloudra/web_launcher.py:154         except OSError:
cloudra/web_launcher.py:155             return False
cloudra/web_launcher.py:156         return True
cloudra/web_launcher.py:157
cloudra/web_launcher.py:158
cloudra/web_launcher.py:159 def _runtime_is_healthy(probe_host: str, port: int) -> bool:
cloudra/web_launcher.py:160     try:
cloudra/web_launcher.py:161         with urllib.request.urlopen(
cloudra/web_launcher.py:162             f"http://{probe_host}:{port}/api/v2/runtime/health", timeout=1.0
cloudra/web_launcher.py:163         ) as resp:
cloudra/web_launcher.py:164             return 200 <= resp.status < 300
cloudra/web_launcher.py:165     except Exception:
cloudra/web_launcher.py:166         return False
cloudra/web_launcher.py:167
cloudra/web_launcher.py:168
cloudra/web_launcher.py:169 def _runtime_serves_spa(probe_host: str, port: int) -> bool:
cloudra/web_launcher.py:170     try:
cloudra/web_launcher.py:171         with urllib.request.urlopen(f"http://{probe_host}:{port}/", timeout=1.0) as resp:
cloudra/web_launcher.py:172             body = resp.read(4096).decode("utf-8", "ignore")
cloudra/web_launcher.py:173             return 200 <= resp.status < 300 and '<div id="root"></div>' in body
cloudra/web_launcher.py:174     except Exception:
cloudra/web_launcher.py:175         return False
cloudra/web_launcher.py:176
cloudra/web_launcher.py:177
cloudra/web_launcher.py:178 def _runtime_present(probe_host: str, port: int) -> bool:
cloudra/web_launcher.py:179     """同端口既提供 API 又提供已构建 SPA，才算可复用的 Cloudra 实例。"""
cloudra/web_launcher.py:180     return _runtime_is_healthy(probe_host, port) and _runtime_serves_spa(probe_host, port)
```

```
cloudra/web_launcher.py:181
cloudra/web_launcher.py:182
cloudra/web_launcher.py:183 def _pinned_runtime(bind_host: str, port: int) -> tuple[int, bool]:
cloudra/web_launcher.py:184     """显式指定的端口：不扫描邻近端口，占用而不可复用时交给 uvicorn 在绑定
cloudra/web_launcher.py:185     阶段报错（而不是悄悄换端口）。"""
cloudra/web_launcher.py:186     if not _may_reuse_runtime(bind_host):
cloudra/web_launcher.py:187         return port, False
cloudra/web_launcher.py:188     return port, _runtime_present(_probe_host(bind_host), port)
cloudra/web_launcher.py:189
cloudra/web_launcher.py:190
cloudra/web_launcher.py:191 def _find_api_port(preferred: int, env_name: str, bind_host: str) -> tuple[int, bool]:
cloudra/web_launcher.py:192     """Return (port, reused_existing_runtime).
cloudra/web_launcher.py:193
cloudra/web_launcher.py:194     A runtime is only reused when it both serves the API and the built SPA on
cloudra/web_launcher.py:195     the same port; otherwise we start a fresh single-port runtime on a free
cloudra/web_launcher.py:196     port so the frontend never points at a stale or foreign server.
cloudra/web_launcher.py:197     """
cloudra/web_launcher.py:198     probe_host = _probe_host(bind_host)
cloudra/web_launcher.py:199     reuse = _may_reuse_runtime(bind_host)
cloudra/web_launcher.py:200     env_value = os.environ.get(env_name)
cloudra/web_launcher.py:201     if env_value:
cloudra/web_launcher.py:202         port = int(env_value)
cloudra/web_launcher.py:203         return port, reuse and _runtime_present(probe_host, port)
cloudra/web_launcher.py:204     if _port_is_bindable(bind_host, preferred):
cloudra/web_launcher.py:205         return preferred, False
cloudra/web_launcher.py:206     if reuse and _runtime_present(probe_host, preferred):
cloudra/web_launcher.py:207         return preferred, True
cloudra/web_launcher.py:208     for port in range(preferred + 1, preferred + 100):
cloudra/web_launcher.py:209         if _port_is_bindable(bind_host, port):
cloudra/web_launcher.py:210             return port, False
cloudra/web_launcher.py:211     raise RuntimeError(f"no free port near {preferred}; set {env_name} explicitly")
cloudra/web_launcher.py:212
cloudra/web_launcher.py:213
cloudra/web_launcher.py:214 def _wait_healthy(url: str, timeout: float = 30.0) -> None:
cloudra/web_launcher.py:215     deadline = time.monotonic() + timeout
cloudra/web_launcher.py:216     while time.monotonic() < deadline:
cloudra/web_launcher.py:217         try:
cloudra/web_launcher.py:218             with urllib.request.urlopen(url, timeout=1.0) as resp:
cloudra/web_launcher.py:219                 if 200 <= resp.status < 300:
cloudra/web_launcher.py:220                     return
cloudra/web_launcher.py:221         except Exception:
cloudra/web_launcher.py:222             pass
cloudra/web_launcher.py:223         time.sleep(0.25)
cloudra/web_launcher.py:224     raise RuntimeError(f"runtime did not become healthy at {url}")
cloudra/web_launcher.py:225
cloudra/web_launcher.py:226
cloudra/web_launcher.py:227 def _terminate_runtime(runtime: subprocess.Popen, *, grace: float = 20.0) -> None:
cloudra/web_launcher.py:228     """SIGTERM → 有界等待 → SIGKILL 升级 → reap。
cloudra/web_launcher.py:229
cloudra/web_launcher.py:230     SIGTERM 让 uvicorn 走优雅关停（lifespan 收尾：停准入 → 取消推理/收尾
cloudra/web_launcher.py:231     任务 → 关流/客户端 → 回收 JS host）；不许只发信号假定已回收（停止
cloudra/web_launcher.py:232     语义契约 6）。
```

```
    """
    if runtime.poll() is not None:
        return
    runtime.terminate()
    try:
        runtime.wait(timeout=grace)
        return
    except subprocess.TimeoutExpired:
        logger.warning("web runtime did not exit within %.0fs of SIGTERM; killing", grace)
    runtime.kill()
    try:
        runtime.wait(timeout=5)
    except subprocess.TimeoutExpired:
        logger.error("web runtime still alive after SIGKILL")


def _primary_address() -> str | None:
    """本机对外的首选 IPv4；取不到（无默认路由）返回 ``None``。

    UDP connect 只让内核选出口网卡、不实际发包；不解析 `ifconfig` 文本。
    仅用于把「别的机器能访问的地址」打印给用户——通配绑定下控制台里的
    回环 URL 复制到另一台机器上没有意义。
    """
    try:
        with socket.socket(socket.AF_INET, socket.SOCK_DGRAM) as sock:
            # 192.0.2.0/24 是 TEST-NET-1，永不路由到真实主机。
            sock.connect(("192.0.2.1", 1))
            return sock.getsockname()[0]
    except OSError:
        return None


def _ui_url(url_host: str, api_port: int, token: str | None) -> str:
    """访问 URL；认证开启时把 token 放 fragment 里（fragment 不进 HTTP 请求，
    前端启动时读入 sessionStorage 后从地址栏抹掉）。"""
    url = f"http://{url_host}:{api_port}"
    return f"{url}/#token={token}" if token is not None else url


def _uvicorn_argv(api_port: int, bind_host: str) -> list[str]:
    """uvicorn 子进程命令行；关停上界见 ``_GRACEFUL_SHUTDOWN_TIMEOUT_SECONDS``。"""
    return [
        sys.executable,
        "-m",
        "uvicorn",
        "cloudra.services.api_v2:create_default_v2_app",
        "--factory",
        "--host",
        bind_host,
        "--port",
        str(api_port),
        "--timeout-graceful-shutdown",
```

```
cloudra/web_launcher.py:285         str(_GRACEFUL_SHUTDOWN_TIMEOUT_SECONDS),
cloudra/web_launcher.py:286         "--log-level",
cloudra/web_launcher.py:287         "warning",
cloudra/web_launcher.py:288     ]
cloudra/web_launcher.py:289
cloudra/web_launcher.py:290
cloudra/web_launcher.py:291 def launch_browser(
cloudra/web_launcher.py:292     frontend: Path,
cloudra/web_launcher.py:293     *,
cloudra/web_launcher.py:294     rebuild: bool,
cloudra/web_launcher.py:295     skip_build: bool,
cloudra/web_launcher.py:296     host: str | None = None,
cloudra/web_launcher.py:297     port: int | None = None,
cloudra/web_launcher.py:298 ) -> int:
cloudra/web_launcher.py:299     bind_host = (host or "").strip() or _DEFAULT_BIND_HOST
cloudra/web_launcher.py:300     probe_host = _probe_host(bind_host)
cloudra/web_launcher.py:301     if port is not None:
cloudra/web_launcher.py:302         api_port, reused_runtime = _pinned_runtime(bind_host, port)
cloudra/web_launcher.py:303     else:
cloudra/web_launcher.py:304         api_port, reused_runtime = _find_api_port(8000, "CLOUDRA_API_PORT", bind_host)
cloudra/web_launcher.py:305     if not _is_loopback_bind(bind_host):
cloudra/web_launcher.py:306         print(
cloudra/web_launcher.py:307             f"[cloudra] warning: binding to non-loopback address {bind_host} — the API is "
cloudra/web_launcher.py:308             "reachable from the network and the bearer token is the only access control",
cloudra/web_launcher.py:309             flush=True,
cloudra/web_launcher.py:310         )
cloudra/web_launcher.py:311     _ensure_frontend(frontend, rebuild=rebuild, skip_build=skip_build)
cloudra/web_launcher.py:312     # 显式启动清除 `cloudra stop` 的持久关停 marker，否则 ensure 拒绝拉起。
cloudra/web_launcher.py:313     from cloudra.services.daemon_supervisor import clear_daemons_shutdown
cloudra/web_launcher.py:314
cloudra/web_launcher.py:315     clear_daemons_shutdown()
cloudra/web_launcher.py:316     ensure_daemons()
cloudra/web_launcher.py:317     from cloudra.persistence import paths
cloudra/web_launcher.py:318
cloudra/web_launcher.py:319     store_path = os.environ.get(
cloudra/web_launcher.py:320         "CLOUDRA_RUNTIME_STORE",
cloudra/web_launcher.py:321         str(paths.runtime_db_path()),
cloudra/web_launcher.py:322     )
cloudra/web_launcher.py:323     env = os.environ.copy()
cloudra/web_launcher.py:324     env["CLOUDRA_RUNTIME_STORE"] = store_path
cloudra/web_launcher.py:325     env["CLOUDRA_FRONTEND_SERVE_DIR"] = str(frontend / "dist")
cloudra/web_launcher.py:326     # 绑定地址传给 app（Host 校验白名单据此放开；绑回环时保持原白名单）。
cloudra/web_launcher.py:327     env["CLOUDRA_API_BIND_HOST"] = bind_host
cloudra/web_launcher.py:328     runtime = None
cloudra/web_launcher.py:329     if reused_runtime:
cloudra/web_launcher.py:330         print(f"[cloudra] reusing existing Cloudra runtime on port {api_port}", flush=True)
cloudra/web_launcher.py:331     else:
cloudra/web_launcher.py:332         print(f"[cloudra] starting web server on {bind_host}:{api_port}...", flush=True)
cloudra/web_launcher.py:333         runtime = subprocess.Popen(
cloudra/web_launcher.py:334             _uvicorn_argv(api_port, bind_host),
cloudra/web_launcher.py:335             env=env,
cloudra/web_launcher.py:336             # 固定子进程 cwd：`python -m` 会把 cwd 放到 sys.path 最前面，
```

```
cloudra/web_launcher.py:337            # cwd 下一个无 __init__.py 的 `cloudra` 目录会把已安装包遮蔽成
cloudra/web_launcher.py:338            # namespace 包（__version__ 缺失、子模块却能被 editable finder
cloudra/web_launcher.py:339            # 加载，报 "unknown location"）。frontend 目录已校验存在且不
cloudra/web_launcher.py:340            # 含 cloudra 子目录，是确定无遮蔽的工作目录。
cloudra/web_launcher.py:341            cwd=frontend,
cloudra/web_launcher.py:342        )
cloudra/web_launcher.py:343    try:
cloudra/web_launcher.py:344        _wait_healthy(f"http://{probe_host}:{api_port}/api/v2/runtime/health")
cloudra/web_launcher.py:345        from cloudra.services.api_token import resolve_api_token
cloudra/web_launcher.py:346
cloudra/web_launcher.py:347        token = resolve_api_token()
cloudra/web_launcher.py:348        shown_url = _ui_url(probe_host, api_port, token)
cloudra/web_launcher.py:349        print(f"[cloudra] web UI: {shown_url}", flush=True)
cloudra/web_launcher.py:350        if bind_host in _WILDCARD_BIND_HOSTS:
cloudra/web_launcher.py:351            # 通配绑定常常是「服务在远端、浏览器在别处」（SSH/局域网）：
cloudra/web_launcher.py:352            # 控制台里的回环 URL 在那台机器上打不开，把对外地址一并给出。
cloudra/web_launcher.py:353            reachable = _primary_address()
cloudra/web_launcher.py:354            if reachable:
cloudra/web_launcher.py:355                print(
cloudra/web_launcher.py:356                    f"[cloudra] reachable from other machines: "
cloudra/web_launcher.py:357                    f"{_ui_url(reachable, api_port, token)}",
cloudra/web_launcher.py:358                    flush=True,
cloudra/web_launcher.py:359                )
cloudra/web_launcher.py:360        if os.environ.get("CLOUDRA_NO_BROWSER", "").strip().lower() not in {"1", "true", "yes
"}:
cloudra/web_launcher.py:361            webbrowser.open(shown_url)
cloudra/web_launcher.py:362        if runtime is None:
cloudra/web_launcher.py:363            return 0
cloudra/web_launcher.py:364        return runtime.wait()
cloudra/web_launcher.py:365    except KeyboardInterrupt:
cloudra/web_launcher.py:366        return 0
cloudra/web_launcher.py:367    finally:
cloudra/web_launcher.py:368        if runtime is not None:
cloudra/web_launcher.py:369            _terminate_runtime(runtime)
```